

Patrones espaciales — ATUS ZMM

Dónde se concentran los accidentes de la Zona Metropolitana de Monterrey, 2019-2024, y qué modelos de agrupamiento dicen algo que no se vea ya en el mapa.

Fuente: `data/processed/atus_zmm_limpio.parquet` (379,294 registros × 71 columnas). Ese archivo **no está versionado** — se genera con:

```
uv run limpiar-atus
```

Las columnas en MAYÚSCULAS son del INEGI; las minúsculas las construye `geostats.limpieza`. Ver [docs/limpieza.md](#).

Qué hay aquí

§	Visual	Unidad
1	El quiebre del geocodificador en 2021	Registro
2	Curva de concentración de accidentes por cruce	Nodo
3	Mapa de puntos negros	Nodo
4	Paneles pequeños por año	Rejilla
5	Perfil de un corredor por cruce	Cruce
6	Hora × día de la semana	Registro
7	Hora × día por tipo de usuario vulnerable	Registro
8	Rejilla hexagonal: proporción con víctimas	Rejilla 500 m
9	Gi* de Getis-Ord: agrupamiento con significancia	Rejilla 500 m
10	DBSCAN: la versión que falla y la que sirve	Nodo
11	Tipología de cruces: <i>k</i> -medias sobre el perfil	Nodo

§	Visual	Unidad
12	Tabla de puntos negros con su tipo	Nodo

La unidad de análisis es el cruce, no el punto. El 96.6 % de los registros tiene `ubicacion == "Intersección"`: esta base no es un patrón de puntos continuo sobre el territorio, es un **conteo sobre nodos de la red viaria**. Una superficie de densidad sobre los registros crudos contesta «¿dónde está la ciudad?»; el conteo por nodo contesta «¿qué cruce es peligroso?», que es la pregunta. Todo el documento agrupa por `id_punto_4d` (10 m de tolerancia) por la razón que demuestra el § 1.

```
import numpy as np
import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap, Normalize
from matplotlib.lines import Line2D
from shapely.geometry import MultiPoint
from scipy import stats
from sklearn.cluster import DBSCAN, KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from statsmodels.stats.multitest import multipletests

from geostats import rutas

d = pd.read_parquet(rutas.ATUS_ZMM_LIMPIO)
N = len(d)
print(f"{N:,} registros x {len(d.columns)} columnas | "
      f"{d.fecha.min():%Y-%m-%d} a {d.fecha.max():%Y-%m-%d} | "
      f"{d.CVE_MUN.nunique()} municipios")
```

379,294 registros x 71 columnas | 2019-01-01 a 2024-12-31 | 18 municipios

Estilo de las gráficas

Misma identidad que `calidad_datos.qmd` y `analisis_multivariado.qmd`. Tres añadidos que este documento necesita:

- **Los mapas no llevan ejes.** Una coordenada proyectada en metros no se lee; lo que orienta es una barra de escala en kilómetros.
- **No hay mapa base ni contornos municipales.** `data/raw/` no trae el marco geoestadístico del INEGI, así que la silueta de la ciudad la dibujan los propios accidentes: los 54,742 nodos en gris claro son el fondo de todos los mapas. Es una limitación, no una decisión de diseño — ver § Limitaciones.
- **Secuencial para magnitud sin signo, divergente solo para polaridad real.** Un conteo y una tasa son magnitudes: un tono, claro a oscuro, nunca arcoíris. La divergente se usa una sola vez, en los perfiles de

la tipología, donde el dato sí tiene signo (arriba o abajo del promedio).

```
# --- Identidad GeoStats -----
AZUL      = "#005991"  # Azul Prusia ajustado -> datos
ROJO      = "#8B2C1A"  # Rojo profundo -> énfasis, títulos
OXIDO     = "#B15E2E"  # Rojo óxido -> detalle cálido
GRAFITO   = "#2C2C2C"  # texto y estructura
GRIS      = "#F2F2F2"  # fondos, rejilla y punto medio de la divergente
BLANCO    = "#FFFFFF"

AZUL_RAMPA = ["#005991", "#1b77b8", "#4195d9"]
FONDO_PTS  = "#DCDCDC" # los nodos como silueta de la ciudad

SECUENCIAL = LinearSegmentedColormap.from_list("geostats_seq", [BLANCO, AZUL])
DIVERGENTE = LinearSegmentedColormap.from_list("geostats_div", [AZUL, GRIS, ROJO])

TITULAR    = ["Montserrat", "Helvetica Neue", "Arial", "DejaVu Sans"]
TEXTO      = ["Cormorant Garamond", "EB Garamond", "Georgia", "DejaVu Serif"]
CIFRAS     = ["Roboto Mono", "Menlo", "DejaVu Sans Mono"]

plt.rcParams.update({
    "figure.dpi": 130, "figure.facecolor": BLANCO, "axes.facecolor": BLANCO,
    "savefig.facecolor": BLANCO, "font.family": "sans-serif",
    "font.sans-serif": TITULAR, "font.serif": TEXTO, "font.monospace": CIFRAS,
    "font.size": 9, "axes.edgecolor": GRIS, "axes.labelcolor": GRAFITO,
    "axes.labelsize": 9, "axes.titlecolor": ROJO, "axes.titlesize": 11.5,
    "axes.titleweight": "bold", "axes.titlelocation": "left", "axes.titlepad": 16,
    "axes.grid": True, "axes.axisbelow": True, "grid.color": GRIS,
    "grid.linewidth": 1.0, "xtick.color": GRAFITO, "ytick.color": GRAFITO,
    "xtick.labelsize": 8.5, "ytick.labelsize": 8.5, "legend.frameon": False,
    "legend.fontsize": 8.5, "lines.linewidth": 1.8,
})

def pulir(ax, ejes=("top", "right", "left")):
    """Quita marco y pone las cifras en monoespaciada."""
    for lado in ejes:
        ax.spines[lado].set_visible(False)
    ax.tick_params(length=0)
    for etiqueta in ax.get_xticklabels() + ax.get_yticklabels():
        etiqueta.set_fontfamily("monospace")
    return ax

def insignias(ax, xs, ys, desde=1):
    """Numera puntos en un mapa con una insignia circular.
```

En la mancha urbana de la ZMM los nombres de vialidad se encabalgan unos sobre otros: son largos y los puntos altos están a menos de 300 m. La insignia numerada nunca choca y la correspondencia vive en la tabla que acompaña a la figura.

```

"""
for i, (x, y) in enumerate(zip(xs, ys), start=desde):
    ax.plot(x, y, "o", ms=13, mfc=BLANCO, mec=ROJO, mew=1.3, zorder=7)
    ax.text(x, y, str(i), ha="center", va="center", fontsize=7,
            weight="bold", color=ROJO, family="monospace", zorder=8)

def lienzo_mapa(ax, extension, escala_km=5, fondo=None):
    """Prepara un eje como mapa: sin ejes, aspecto 1:1 y barra de escala."""
    if fondo is not None:
        ax.scatter(fondo[0], fondo[1], s=0.35, c=FONDO_PTS, linewidths=0, zorder=1)
    ax.set_xlim(extension[0], extension[1])
    ax.set_ylim(extension[2], extension[3])
    ax.set_aspect("equal")
    ax.grid(False)
    ax.set_axis_off()
    x0 = extension[0] + 0.05 * (extension[1] - extension[0])
    y0 = extension[2] + 0.06 * (extension[3] - extension[2])
    ax.plot([x0, x0 + escala_km * 1000], [y0, y0], color=GRAFITO, lw=1.6,
            solid_capstyle="butt", zorder=9)
    ax.text(x0, y0 + 0.012 * (extension[3] - extension[2]), f"{escala_km} km",
            fontsize=7.5, color=GRAFITO, family="monospace", va="bottom", zorder=9)
    return ax

```

1. El quiebre del geocodificador en 2021

Antes de cualquier mapa hay que resolver un problema que invalida la comparación entre años. El INEGI **cambió la precisión con la que geocodifica** a partir de 2021: 2019 y 2020 traen coordenadas con cinco decimales o menos, y de 2021 en adelante casi todo viene con seis a ocho.

```

decimales = d.LONGITUD.astype(str).str.split(".").str[1].str.len()
regimen = np.where(decimales <= 5, "<= 5 decimales", ">= 6 decimales")
tabla = pd.crosstab(d.ANIO, regimen, normalize="index").mul(100).round(1)
print(tabla.to_string())

```

col_0	≤ 5 decimales	≥ 6 decimales
ANIO		
2019	100.0	0.0
2020	100.0	0.0
2021	0.8	99.2
2022	1.0	99.0
2023	1.1	98.9
2024	1.0	99.0

El efecto sobre los datos es directo: conforme sube la precisión, cada vez menos registros caen exactamente sobre la misma coordenada. Eso **parece** dispersión de los accidentes y no lo es.

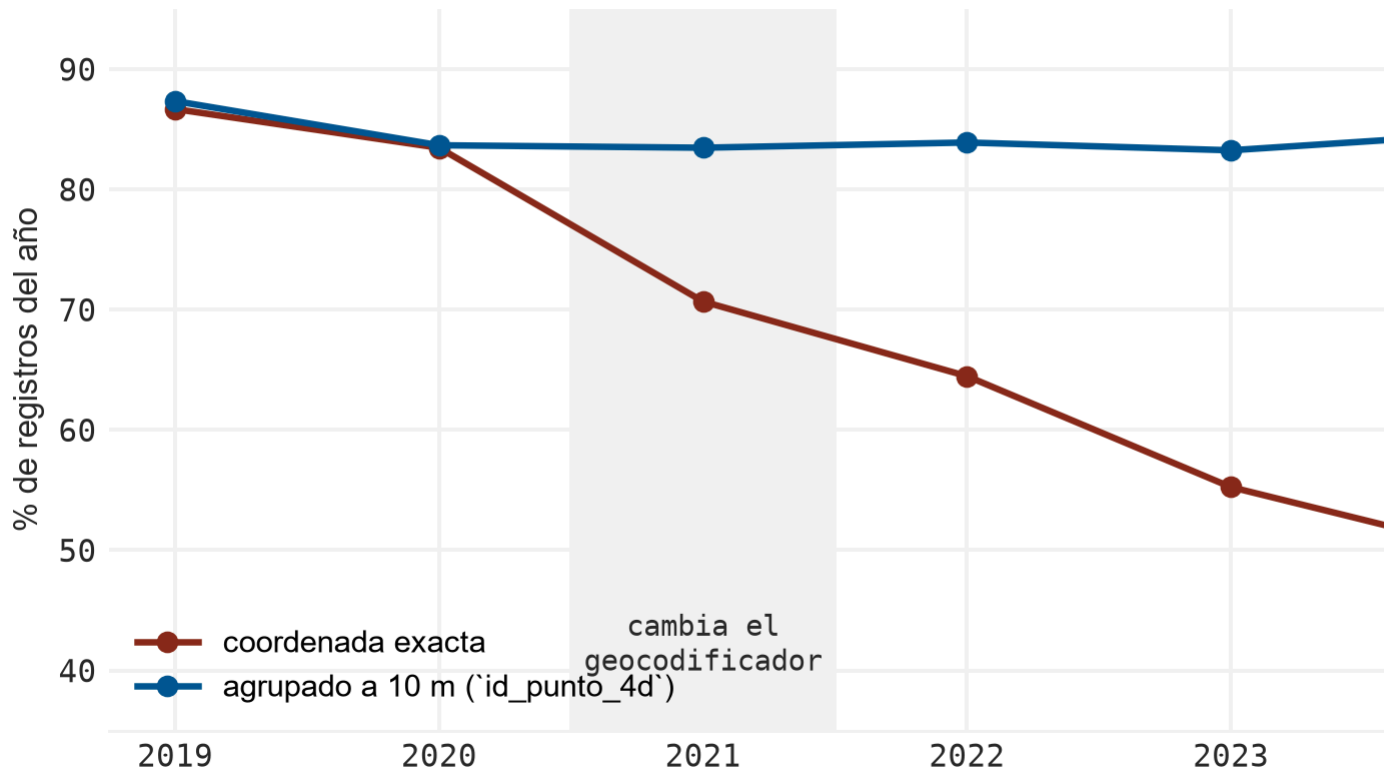
```

por_anio = pd.DataFrame({
    "exacta": d.groupby("ANIO").id_punto.apply(lambda s: 100 * s.duplicated(keep=False).mean()),
    "10 m": d.groupby("ANIO").id_punto_4d.apply(lambda s: 100 * s.duplicated(keep=False).mean())
})

fig, ax = plt.subplots(figsize=(7.2, 3.6))
ax.plot(por_anio.index, por_anio["exacta"], color=ROJO, marker="o", ms=5,
        label="coordenada exacta")
ax.plot(por_anio.index, por_anio["10 m"], color=AZUL, marker="o", ms=5,
        label="agrupado a 10 m (`id_punto_4d`)")
ax.axvspan(2020.5, 2021.5, color=GRIS, zorder=0)
ax.text(2021, 40, "cambia el\ngecocodificador", fontsize=8, color=GRAFITO,
        ha="center", family="monospace")
for serie, color, dy in (("exacta", ROJO, -6), ("10 m", AZUL, 3)):
    ax.annotate(f"{por_anio[serie].iloc[-1]:.0f} %",
               (por_anio.index[-1], por_anio[serie].iloc[-1]),
               textcoords="offset points", xytext=(6, dy), color=color,
               fontsize=8.5, family="monospace", weight="bold")
ax.set_title("Registros que comparten ubicación con otro, por año")
ax.set_ylabel("% de registros del año")
ax.set_ylim(35, 95)
ax.set_xticks(por_anio.index)
ax.legend(loc="lower left")
pulir(ax)
plt.show()

```

Registros que comparten ubicación con otro, por año



La coordenada exacta cae de 86.7 % a 49.8 % en seis años. Agrupada a 10 m se queda plana entre 83 % y 85 %, todos los años. La conclusión es la que gobierna el resto del documento:

La dispersión aparente es del geocodificador, no de los accidentes. Cualquier mapa de densidad o conteo comparado entre años sobre la coordenada cruda mide un cambio de método administrativo. Sobre `id_punto_4d` la serie es comparable, porque los 10 m de tolerancia absorben la diferencia de precisión.

Esta es una corrección a [docs/limpieza.md](#), que reporta `precision_baja` como un problema de 3 registros. Por ambos ejes son 3, sí — pero el problema real no es el registro individual con pocos decimales, es que **el régimen de precisión es una función del año**.

0. La tabla de nodos

Un nodo es una celda de 10 m con al menos un accidente. Su coordenada representativa es la mediana de los registros que caen en ella, y su etiqueta el par de vialidades más frecuente. Todo se proyecta a **EPSG:6372** (Cónica Conforme de Lambert para México) para que las distancias estén en metros y `eps` de DBSCAN o el lado de un hexágono signifiquen algo.

```

g = d.groupby("id_punto_4d", observed=True, sort=False)
nodos = pd.DataFrame({"n": g.size(), "lon": g.LONGITUD.median(), "lat": g.LATITUD.median()})

etiquetas = (d.groupby(["id_punto_4d", "NOM_MUN", "calle1_norm", "calle2_norm"],
                      observed=True, dropna=False, sort=False).size()
              .rename("f").reset_index()
              .sort_values("f", ascending=False)
              .drop_duplicates("id_punto_4d")
              .set_index("id_punto_4d"))
nodos = nodos.join(etiquetas[["NOM_MUN", "calle1_norm", "calle2_norm"]])

proyectado = gpd.GeoSeries(gpd.points_from_xy(nodos.lon, nodos.lat), crs=4326).to_crs(6372)
nodos["x"], nodos["y"] = proyectado.x.values, proyectado.y.values
nodos = nodos.sort_values("n", ascending=False)

# Extensión de trabajo: el 99 % central de los accidentes. La ZMM formal mide
# 105 x 120 km porque incluye Santiago, Hidalgo y Salinas Victoria; encuadrar
# ahí deja la conurbación en una esquina.
px = np.repeat(nodos.x.to_numpy(), nodos.n.to_numpy())
py = np.repeat(nodos.y.to_numpy(), nodos.n.to_numpy())
EXT = (*np.percentile(px, [1, 99]), *np.percentile(py, [1, 99]))
FONDO = (nodos.x.to_numpy(), nodos.y.to_numpy())

print(f"{len(nodos):,} nodos de 10 m | "
      f"encuadre {(EXT[1]-EXT[0])/1000:.0f} x {(EXT[3]-EXT[2])/1000:.0f} km")
print(nodos.head(5)[["n", "NOM_MUN", "calle1_norm", "calle2_norm"]].to_string())

```

54,742 nodos de 10 m | encuadre 51 x 44 km

	n	NOM_MUN	calle1_norm	calle2_norm
id_punto_4d				
-100.3093,25.7307	1024	San Nicolás de los Garza	FRAY LUIS DE LEON	FIDEL VELAZQUEZ
-100.2711,25.6136	1004	Monterrey	EUGENIO GARZA SADA	LAZARO CARDENAS
-100.2143,25.7241	986	San Nicolás de los Garza	ACAPULCO	CARRETERA MIGUEL ALEMAN
-100.3165,25.7303	917	San Nicolás de los Garza	MANUEL L. BARRAGAN	MANUEL L. BARRAGAN
-100.3435,25.6963	810	Monterrey	SIMON BOLIVAR	PASEO DE LOS LEONES

2. La concentración es el hallazgo

54,742 cruces, 379,294 accidentes. Si estuvieran repartidos parejo no habría nada que mapear. La curva de concentración mide qué tan lejos está el reparto de la igualdad.

```

cuenta = nodos.n.to_numpy()
cum_acc = np.cumsum(cuenta) / cuenta.sum()
cum_nodos = np.arange(1, len(cuenta) + 1) / len(cuenta)

fig, ax = plt.subplots(figsize=(5.4, 5.0))
ax.plot([0, 100], [0, 100], color=GRAFITO, lw=1.0, ls=(0, (4, 3)), alpha=0.55)

```

```

ax.plot(cum_nodos * 100, cum_acc * 100, color=AZUL, lw=2.2)
ax.text(80, 74, "reparto uniforme", fontsize=8, color=GRAFITO, rotation=39,
        family="monospace", alpha=0.75)

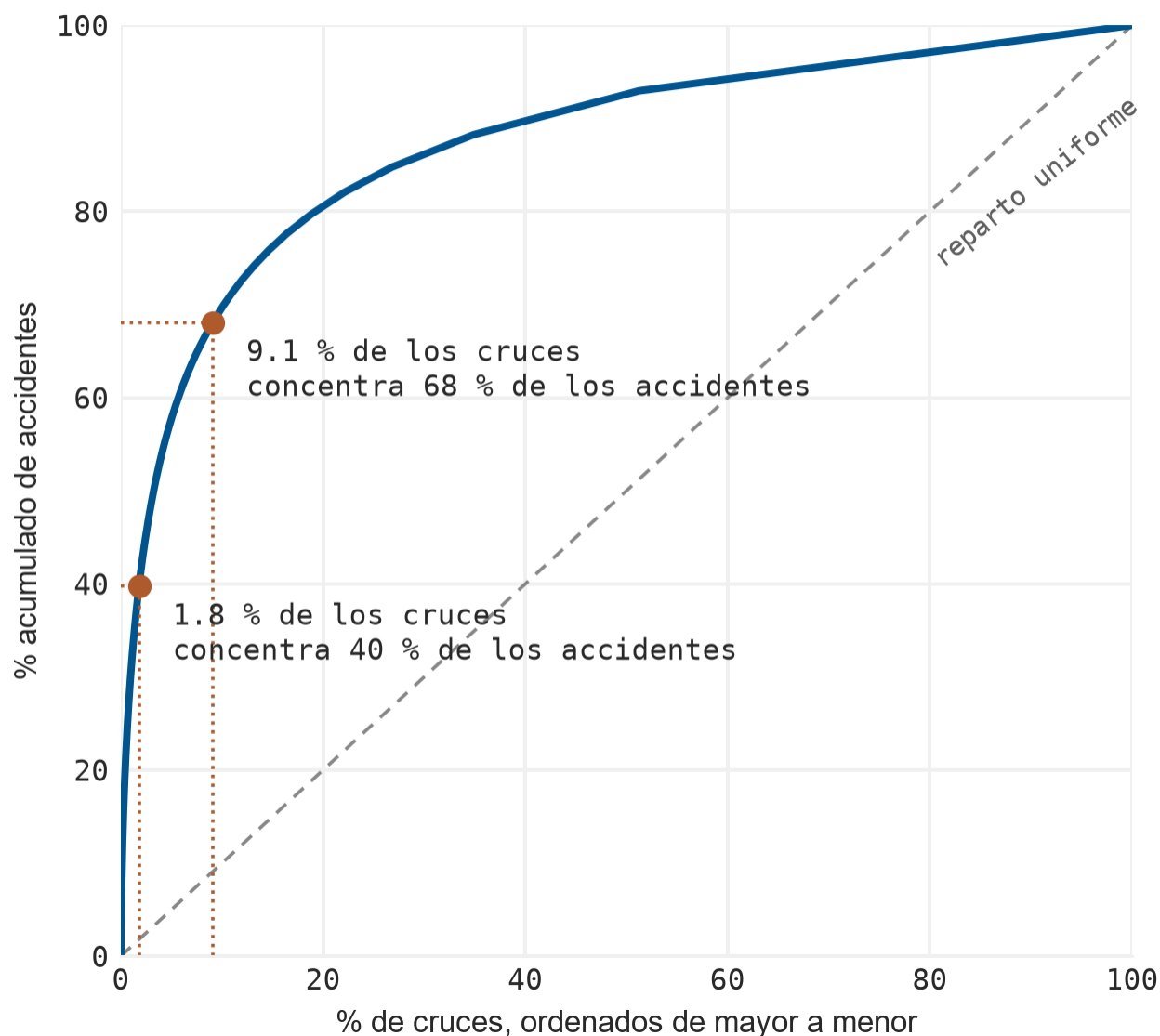
for pct_nodos in (1.8, 9.1):
    k = int(round(pct_nodos / 100 * len(cuenta)))
    y = cum_acc[k - 1] * 100
    ax.plot([pct_nodos, pct_nodos], [0, y], color=OXIDO, lw=1.0, ls=":")
    ax.plot([0, pct_nodos], [y, y], color=OXIDO, lw=1.0, ls=":")
    ax.plot(pct_nodos, y, "o", ms=6, color=OXIDO, zorder=5)
    ax.annotate(f"{pct_nodos} % de los cruces\nconcentra {y:.0f} % de los accidentes",
                (pct_nodos, y), textcoords="offset points", xytext=(10, -4),
                fontsize=8.5, color=GRAFITO, family="monospace", va="top")

ax.set_title("Concentración de accidentes por cruce, ZMM 2019-2024")
ax.set_xlabel("% de cruces, ordenados de mayor a menor")
ax.set_ylabel("% acumulado de accidentes")
ax.set_xlim(0, 100)
ax.set_ylim(0, 100)
pulir(ax)
plt.show()

resumen = pd.DataFrame({
    "cruces": [50, 100, 250, 500, 1000, 2500, 5000],
})
resumen["% de cruces"] = (100 * resumen.cruces / len(cuenta)).round(1)
resumen["% de accidentes"] = [round(100 * cuenta[:k].sum() / cuenta.sum(), 1)
                              for k in resumen.cruces]
print(resumen.to_string(index=False))

```

Concentración de accidentes por cruce, ZMM 2019-2024



cruces	% de cruces	% de accidentes
50	0.1	7.5
100	0.2	12.0
250	0.5	20.6
500	0.9	29.2
1000	1.8	40.0
2500	4.6	56.1
5000	9.1	68.2

El 1.8 % de los cruces concentra el 40 % de los accidentes; el 9.1 %, el 68 %. Esa desigualdad es lo que hace que valga la pena un mapa de puntos negros en lugar de una política de cobertura uniforme.

¿Son los mismos cruces todos los años?

Un ranking de conteos siempre produce una cola alta, aunque el proceso sea puramente aleatorio: el cruce que salió alto un año baja al siguiente por pura reversión a la media. La única defensa es medir persistencia.

```

top200 = nodos.index[:200]
anios_presente = (d[d.id_punto_4d.isin(top200)]
                  .groupby("id_punto_4d", observed=True).ANIO.nunique()
                  .value_counts().sort_index())

temprano = d[d.ANIO.isin([2019, 2020]).id_punto_4d.value_counts()
tardio = d[d.ANIO.isin([2023, 2024]).id_punto_4d.value_counts()
comunes = temprano.index.intersection(tardio.index)
rho = temprano[comunes].corr(tardio[comunes], method="spearman")

print("De los 200 cruces con más accidentes, en cuántos de los 6 años aparecen:")
print(anios_presente.to_string())
print(f"\nSolape de los 100 primeros, 2019-20 vs 2023-24: "
      f"{len(set(temprano.head(100).index) & set(tardio.head(100).index))} de 100")
print(f"Correlación de rangos de Spearman sobre {len(comunes):,} cruces comunes: {rho:.3f}")

```

De los 200 cruces con más accidentes, en cuántos de los 6 años aparecen:

ANIO

2	2
3	2
4	15
5	26
6	155

Solape de los 100 primeros, 2019-20 vs 2023-24: 53 de 100

Correlación de rangos de Spearman sobre 10,114 cruces comunes: 0.624

155 de los 200 aparecen los seis años y la correlación de rangos entre el primer y el último bienio es 0.62.

No es ruido: son los mismos cruces. Con esto el mapa de la sección siguiente es una afirmación sostenible y no un artefacto del muestreo.

3. Mapa de puntos negros

Burbuja por cruce, área proporcional al conteo. Sin superficie de densidad: el dato es discreto y se muestra discreto. Los 54,742 nodos en gris dibujan la ciudad; los 300 primeros, en azul, son el objeto de interés.

```

TOP = 300
pn = nodos.head(TOP)

fig, ax = plt.subplots(figsize=(8.8, 7.4))
lienzo_mapa(ax, EXT, escala_km=5, fondo=FONDO)
ax.scatter(pn.x, pn.y, s=pn.n / 3.4, c=AZUL, alpha=0.5,
           edgecolors=BLANCO, linewidths=0.5, zorder=4)
insignias(ax, nodos.x[:10], nodos.y[:10])

for muestra in (100, 400, 1000):
    ax.scatter([], [], s=muestra / 3.4, c=AZUL, alpha=0.5, edgecolors=BLANCO,

```

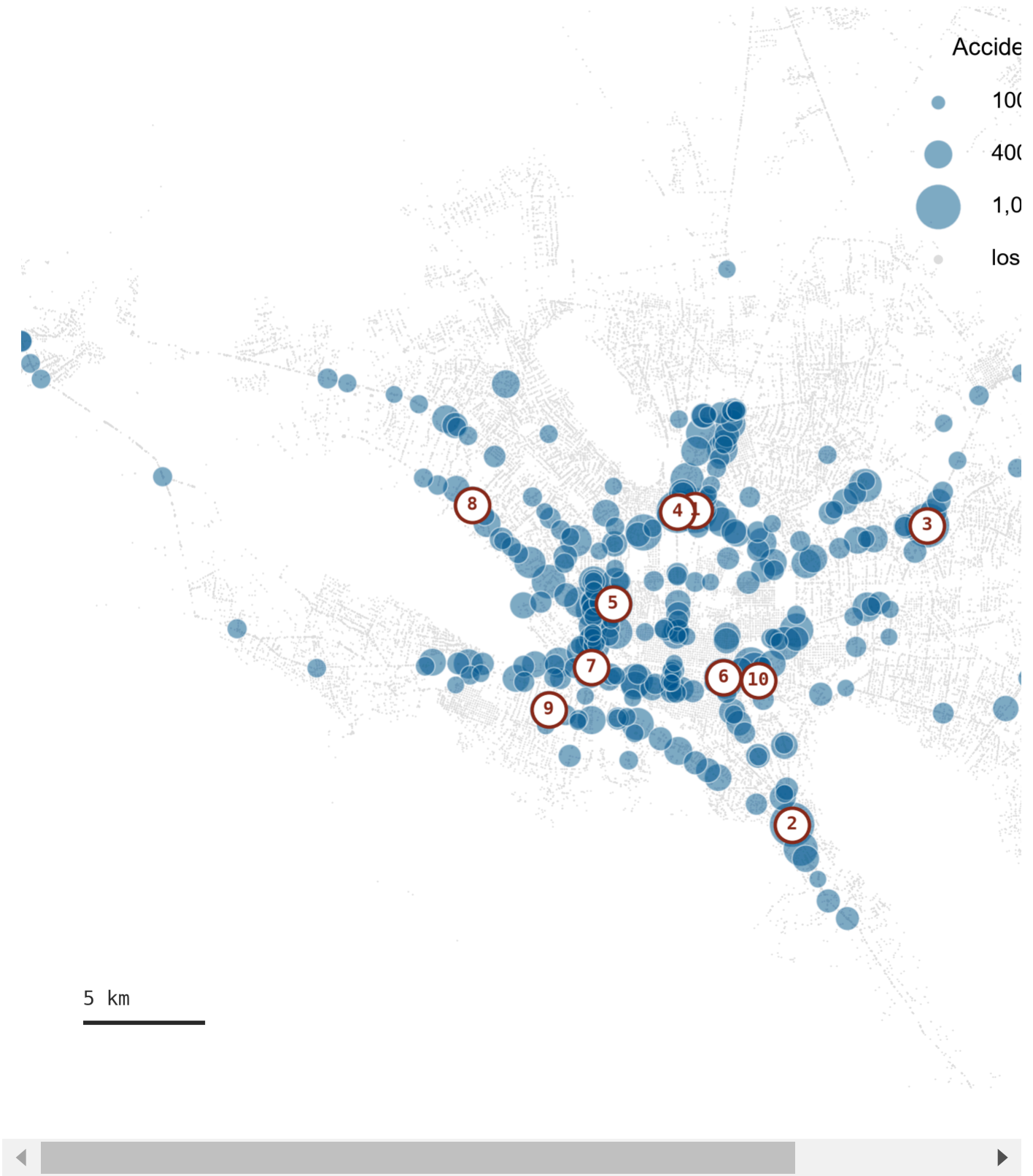
```

        linewidths=0.5, label=f"{muestra:,} accidentes")
ax.scatter([], [], s=6, c=FONDO_PTS, label=f"los otros {len(nodos)-TOP:,} cruces")
ax.legend(loc="upper right", labelspace=1.4, borderpad=0.9,
        handletextpad=1.4, title="Accidentes 2019-2024")
ax.set_title(f"Los {TOP} cruces con más accidentes, ZMM 2019-2024\n"
        "numerados, los 10 primeros", pad=10)
plt.show()

leyenda = nodos.head(10).reset_index(drop=True)
leyenda.index += 1
leyenda["cruce"] = (leyenda.calle1_norm.str.title() + " x "
        + leyenda.calle2_norm.str.title())
print(leyenda[["cruce", "NOM_MUN", "n"]].
        .rename(columns={"NOM_MUN": "municipio", "n": "accidentes"}).to_string())

```

Los 300 cruces con más accidentes, ZMM 2019-2024 numerados, los 10 primeros



cruce municipio

accidentes

1 Fray Luis De Leon x Fidel Velazquez San Nicolás de los Garza

2	Eugenio Garza Sada × Lazaro Cardenas	Monterrey
1004		
3	Acapulco × Carretera Miguel Aleman	San Nicolás de los Garza
986		
4	Manuel L. Barragan × Manuel L. Barragan	San Nicolás de los Garza
917		
5	Simon Bolivar × Paseo De Los Leones	Monterrey
810		
6	Felix Uresti Gomez × Constitucion	Monterrey
761		
7	Avenida San Jeronimo × Avenida Doctor Jose Eleuterio Gonzalez	Monterrey
706		
8	Paseo De Los Leones × Pedro Infante	Monterrey
698		
9	San Pedro × Calzada Del Valle	San Pedro Garza García
698		
10	Revolucion × Chapultepec	Monterrey
693		

El patrón es de **corredores, no de manchas**: Garza Sada, Lincoln-Ruiz Cortines, Gonzalitos, Miguel Alemán y Manuel L. Barragán aparecen como hileras de nodos grandes. Eso es lo que el § 10 intenta recuperar con un modelo.

4. Los seis años, mismo encuadre y misma escala

Paneles pequeños en lugar de seis colores: la comparación es entre panel y panel, no entre series dentro de un panel. La rampa y el máximo son compartidos — sin eso, seis paneles normalizados cada uno a su propio máximo mienten.

```
LADO = 500.0
malla = int((EXT[1] - EXT[0]) / LADO)

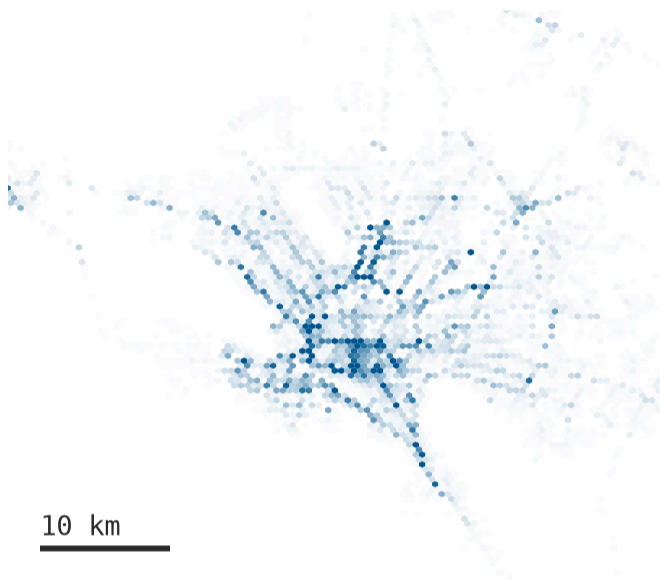
fig, ejes = plt.subplots(2, 3, figsize=(11.5, 8.0))
capas = []
for ax, anio in zip(ejes.ravel(), sorted(d.ANIO.unique())):
    sub = d[d.ANIO == anio].join(nodos[["x", "y"]], on="id_punto_4d")
    capas.append(ax.hexbin(sub.x, sub.y, gridsize=malla, extent=EXT,
                           cmap=SECUENCIAL, mincnt=1, linewidths=0, zorder=3))
    lienzo_mapa(ax, EXT, escala_km=10)
    ax.set_title(f"{anio} · {len(sub):,} accid.", fontsize=10, color=GRAFITO,
                loc="left", pad=6)

# El máximo absoluto lo fija una sola celda del centro y deja el resto casi en
# blanco: se recorta al percentil 99 y la barra se marca como truncada.
vmax = np.percentile(np.concatenate([np.asarray(c.get_array()) for c in capas]), 99)
for c in capas:
    c.set_norm(Normalize(0, vmax))
```

```
fig.suptitle("Accidentes por celda de 500 m, un panel por año",  
            color=ROJO, size=11.5, weight="bold", x=0.09, ha="left", y=0.97)  
barra = fig.colorbar(capas[0], ax=ejes, fraction=0.02, pad=0.02, extend="max")  
barra.set_label("accidentes por celda", size=8.5)  
barra.outline.set_visible(False)  
plt.show()
```

Accidentes por celda de 500 m, un panel por año

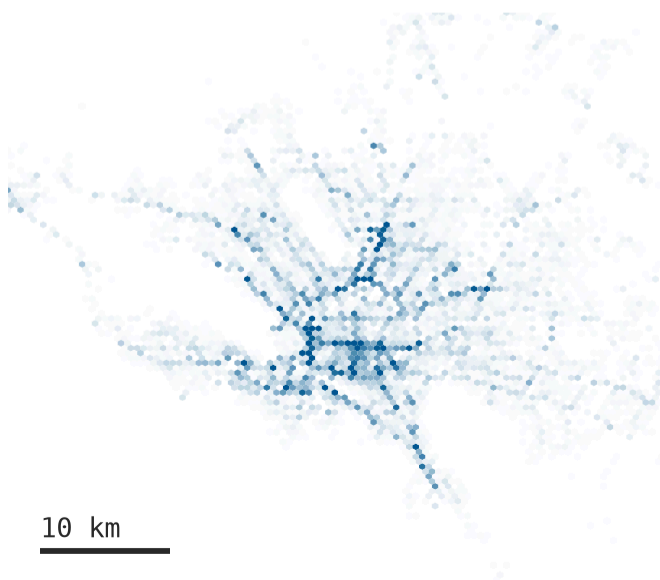
2019 · 63,134 accid.



2020 · 54,041 accid.



2022 · 65,257 accid.



2023 · 69,902 accid.



Los seis paneles se parecen entre sí, y eso es el resultado: la geografía del riesgo en la ZMM es **estable**. Lo que cambia es el volumen (54,041 en 2020 por el confinamiento, 71,321 en 2024), no el lugar.

5. Perfil de un corredor

Un mapa dice «esta avenida está mal». Un perfil por cruce dice «estos cuatro cruces están mal», que ya es accionable. Se elige la vialidad con más accidentes contando las dos columnas de calle, y se desagrega por la vialidad que cruza.

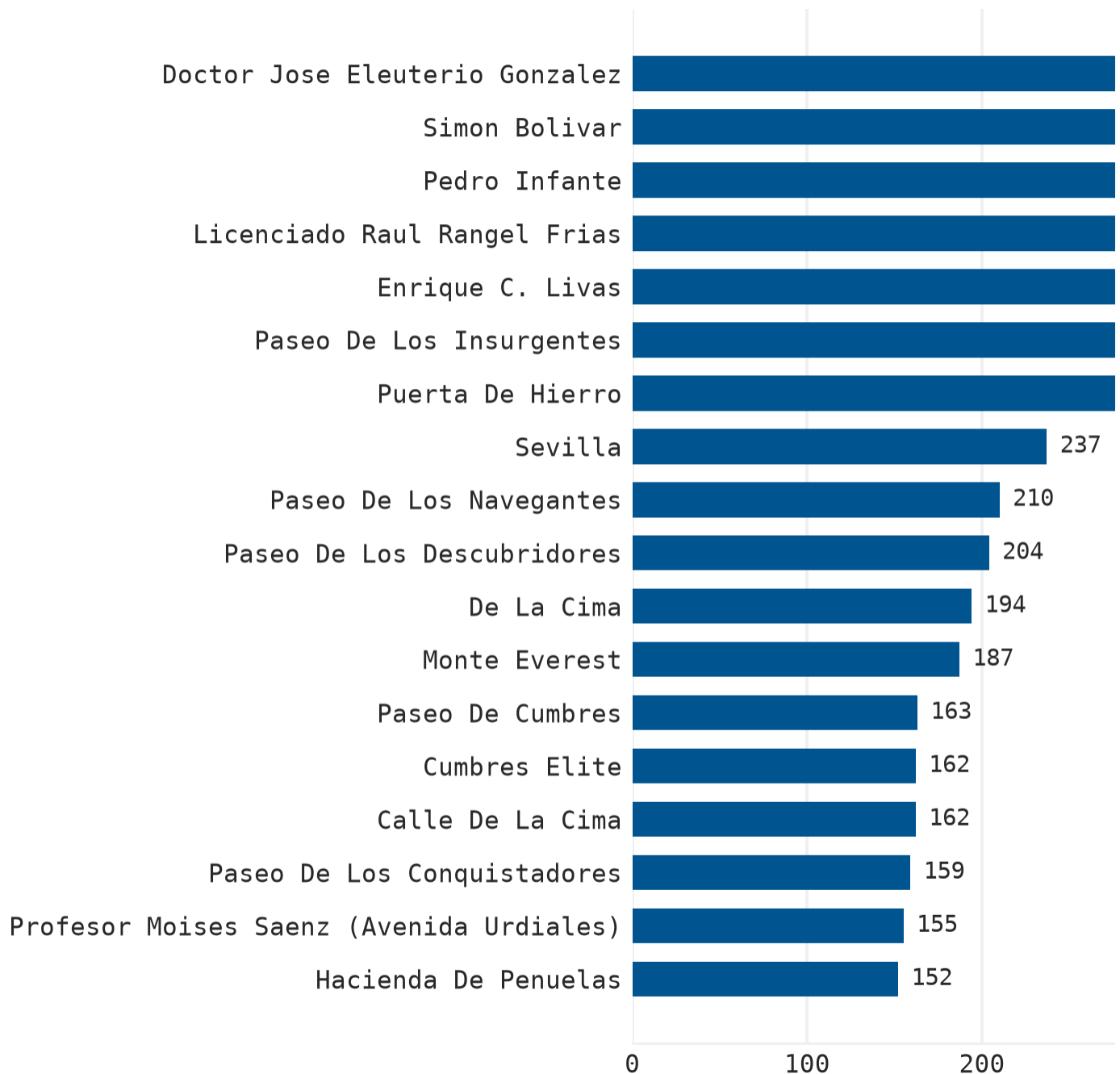
```
vialidades = pd.concat([d.calle1_norm, d.calle2_norm]).value_counts()
print(vialidades.head(8).to_string())

AVENIDA = vialidades.index[0]
en_avenida = d[(d.calle1_norm == AVENIDA) | (d.calle2_norm == AVENIDA)].copy()
en_avenida["cruza"] = np.where(en_avenida.calle1_norm == AVENIDA,
                              en_avenida.calle2_norm, en_avenida.calle1_norm)
perfil = (en_avenida[en_avenida.cruza.notna() & (en_avenida.cruza != AVENIDA)]
          .groupby("cruza").agg(accidentes=("ID", "size"),
                              victimas=("hay_victimitas", "sum"))
          .sort_values("accidentes", ascending=False).head(18).iloc[::-1])

fig, ax = plt.subplots(figsize=(7.6, 6.4))
ax.barh(perfil.index.str.title(), perfil.accidentes, color=AZUL, height=0.66)
for nombre, r in perfil.iterrows():
    ax.text(r.accidentes + max(perfil.accidentes) * 0.012, nombre.title(),
            f"{r.accidentes:},", va="center", fontsize=8, color=GRAFITO,
            family="monospace")
ax.set_title(f"{AVENIDA.title():} accidentes por vialidad que cruza\n"
            f"{len(en_avenida):}, accidentes en total, 2019-2024")
ax.set_xlabel("accidentes 2019-2024")
ax.set_xlim(0, max(perfil.accidentes) * 1.12)
ax.grid(axis="y", visible=False)
pulir(ax, ejes=("top", "right", "left"))
plt.show()
```

PASEO DE LOS LEONES	8315
DOCTOR JOSE ELEUTERIO GONZALEZ	7584
ABRAHAM LINCOLN	7402
EUGENIO GARZA SADA	6995
DOCTOR IGNACIO MORONES PRIETO	6916
ADOLFO RUIZ CORTINES	6487
CONSTITUCION	6357
LAZARO CARDENAS	6054

Paseo De Los Leones: accidentes en total, 2019



6. Hora × día de la semana

`hora` y `día_semana` no tienen un solo nulo tras la limpieza — lujo raro en esta base. 168 celdas, una rampa de un tono: lo único que hay que leer es el orden.

```

DIAS = list(d.dia_semana.cat.categories)

def rejilla_horaria(datos):
    t = (datos.pivot_table(index="dia_semana", columns="hora", values="ID",
                           aggfunc="size", observed=True)
         .reindex(index=DIAS, columns=range(24)).fillna(0))
    return t

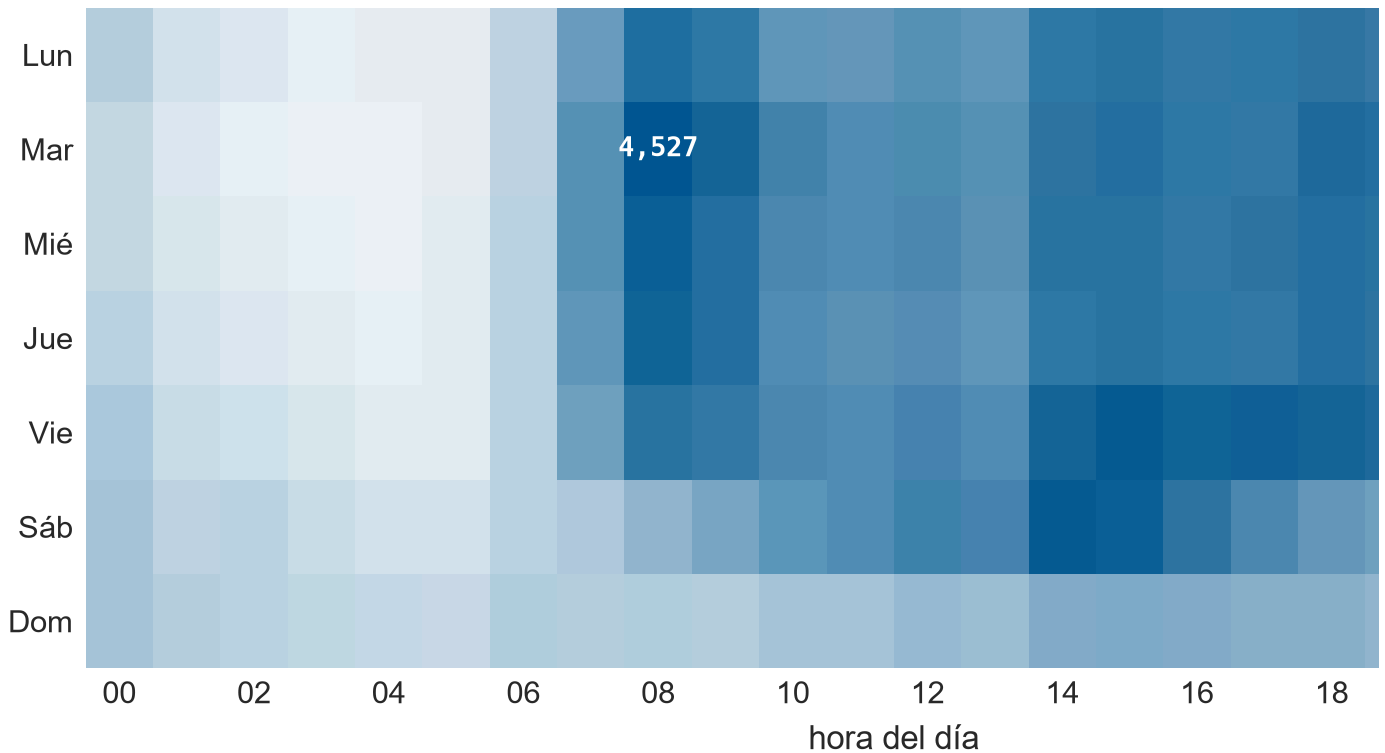
def dibujar_rejilla(ax, t, vmax, etiqueta_max=True):
    im = ax.imshow(t.to_numpy(), cmap=SECUENCIAL, aspect="auto", vmin=0, vmax=vmax)
    ax.set_xticks(range(0, 24, 2), [f"{h:02d}" for h in range(0, 24, 2)])
    ax.set_yticks(range(7), [dia[:3] for dia in DIAS])
    ax.grid(False)
    for lado in ax.spines.values():
        lado.set_visible(False)
    ax.tick_params(length=0)
    if etiqueta_max:
        i, j = np.unravel_index(np.argmax(t.to_numpy()), t.shape)
        ax.text(j, i, f"{t.to_numpy()[i, j]:.0f}", ha="center", va="center",
               fontsize=7.5, color=BLANCO, family="monospace", weight="bold")
    return im

t = rejilla_horaria(d)
fig, ax = plt.subplots(figsize=(8.4, 3.3))
im = dibujar_rejilla(ax, t, t.to_numpy().max())
ax.set_title("Accidentes por hora y día de la semana, ZMM 2019-2024")
ax.set_xlabel("hora del día")
barra = fig.colorbar(im, ax=ax, fraction=0.025, pad=0.02)
barra.set_label("accidentes", size=8.5)
barra.outline.set_visible(False)
plt.show()

pico = t.stack().idxmax()
print(f"Celda máxima: {pico[0]} a las {pico[1]:02d} h con {t.stack().max():.0f} accidentes")
print(f"Madrugada (00-05 h): {100*d.hora.between(0,5).mean():.1f} % de los accidentes")

```

Accidentes por hora y día de la semana, ZMM 2019-2024



Celda máxima: Martes a las 08 h con 4,527 accidentes
Madrugada (00-05 h): 8.6 % de los accidentes

7. La misma rejilla, por tipo de usuario

El agregado esconde a quien más pierde en un accidente. Tres paneles, cada uno en **porcentaje de los accidentes de ese usuario**, con la misma escala: así se comparan perfiles y no volúmenes, que difieren en un orden de magnitud.

```
USUARIOS = [(10, "Motocicleta"), (2, "Peatón atropellado"), (11, "Ciclista")]

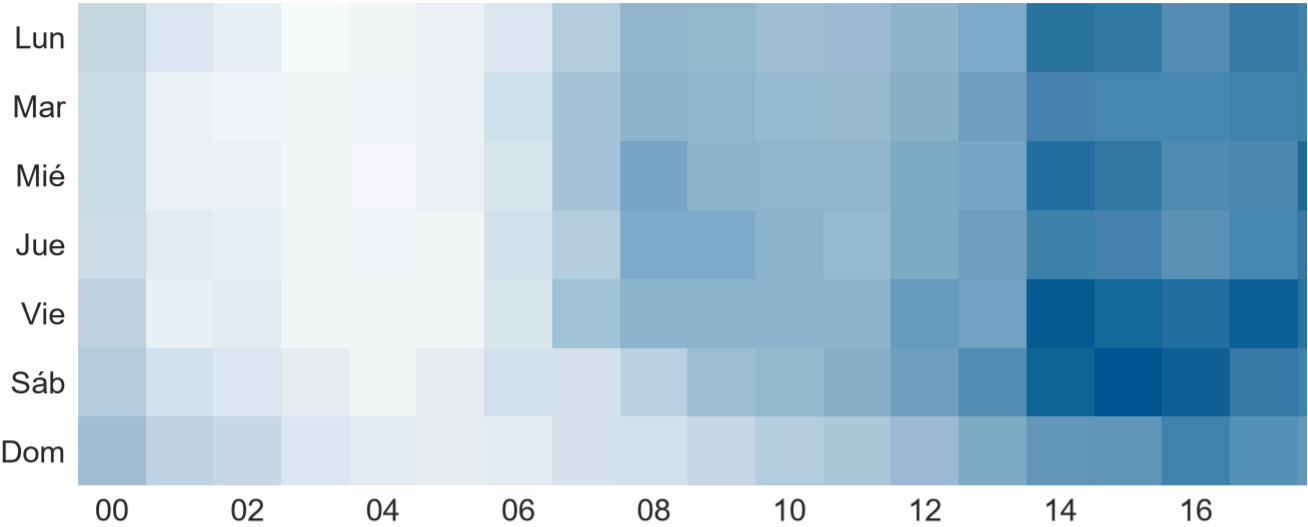
rejillas = []
for codigo, nombre in USUARIOS:
    sub = d[d.TIPACCID == codigo]
    rejillas.append((nombre, len(sub), 100 * rejilla_horaria(sub) / len(sub)))

vmax = max(t.to_numpy().max() for _, _, t in rejillas)
fig, ejes = plt.subplots(3, 1, figsize=(8.4, 8.2))
for ax, (nombre, n, t) in zip(ejes, rejillas):
    im = dibujar_rejilla(ax, t, vmax, etiqueta_max=False)
    ax.set_title(f"{nombre} · {n:,} accidentes", fontsize=10, color=GRAFITO,
                loc="left", pad=6)
ejes[-1].set_xlabel("hora del día")
```

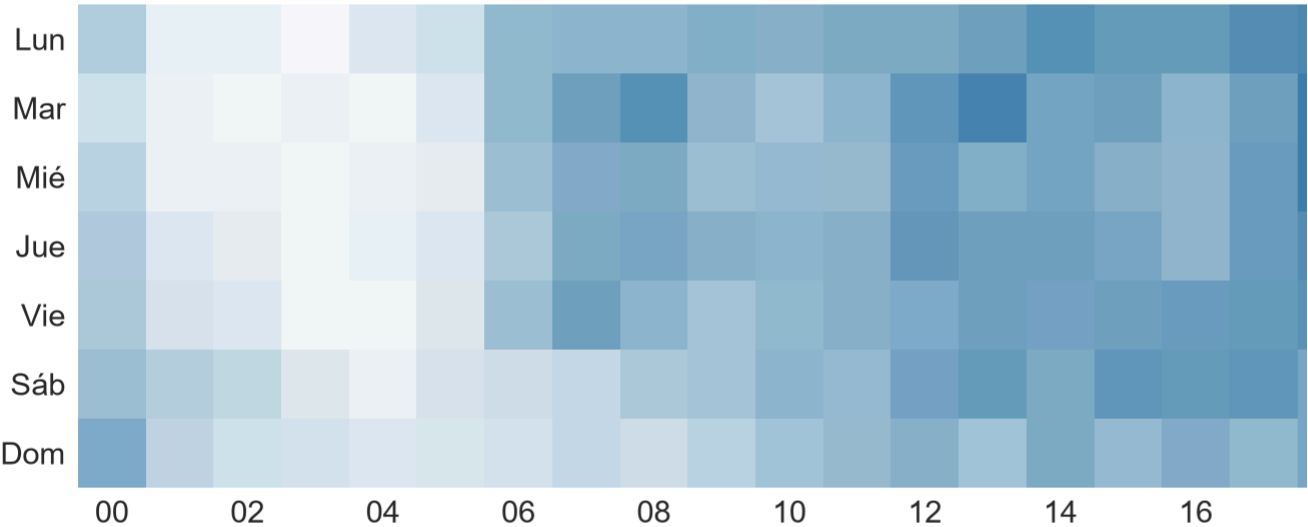
```
fig.suptitle("Perfil horario por tipo de usuario, en % de sus propios accidentes",  
            color=ROJO, size=11.5, weight="bold", x=0.06, ha="left", y=0.965)  
barra = fig.colorbar(im, ax=ejes, fraction=0.02, pad=0.02)  
barra.set_label("% de los accidentes del usuario", size=8.5)  
barra.outline.set_visible(False)  
plt.show()
```

Perfil horario por tipo de usuario, en % de sus propios accidentes

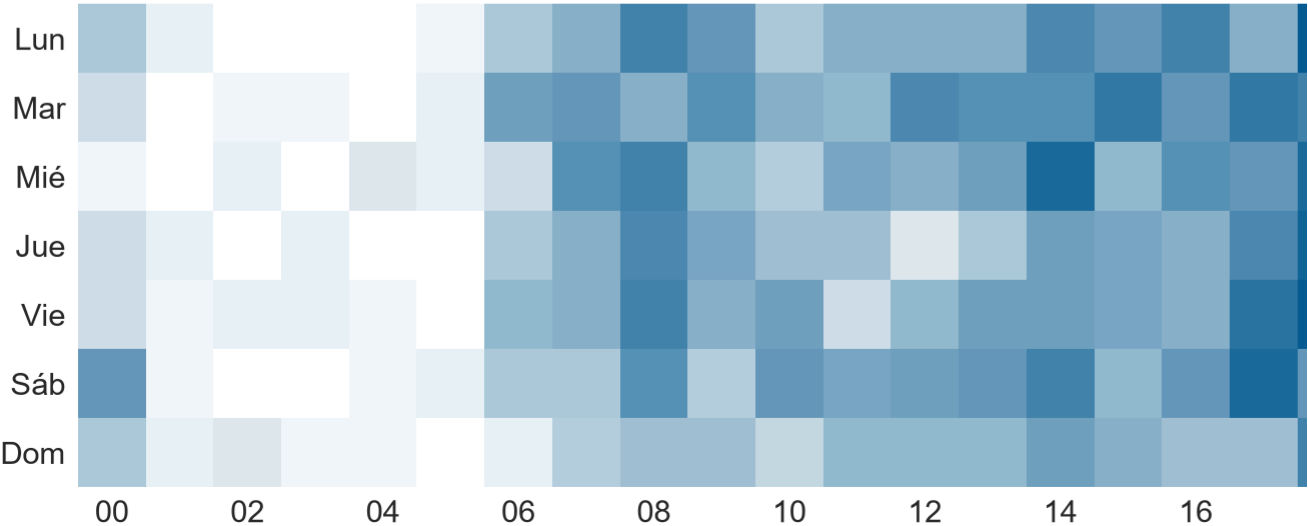
Motocicleta · 24,278 accidentes



Peatón atropellado · 8,553 accidentes



Ciclista · 1,511 accidentes



hora del día

8. Rejilla hexagonal: proporción con víctimas

Aquí cambia la pregunta: no *dónde hay más accidentes* sino **dónde el accidente sale más caro**. Eso es una tasa, y una tasa necesita un piso de denominador. Una celda con 3 accidentes y 1 con víctimas da 33 % y no significa nada, así que las celdas con menos de 30 accidentes se suprimen en lugar de pintarse.

```
acc = d.join(nodos[["x", "y"]], on="id_punto_4d")
PISO = 30

fig, ax = plt.subplots(figsize=(8.6, 8.2))
lienzo_mapa(ax, EXT, escala_km=5, fondo=FONDO)
hb = ax.hexbin(acc.x, acc.y, C=acc.hay_victimas.astype(float),
               reduce_C_function=np.mean, gridsize=malla, extent=EXT,
               cmap=SECUENCIAL, mincnt=PISO, linewidths=0, zorder=3)
tasa = np.asarray(hb.get_array()) * 100
hb.set_array(tasa)
hb.set_clim(0, np.percentile(tasa, 98))
celdas_con_dato = len(ax.hexbin(acc.x, acc.y, gridsize=malla, extent=EXT,
                               mincnt=1, visible=False).get_array())
ax.set_title(f"Proporción de accidentes con víctimas por celda de {LADO:.0f} m\n"
            f"{len(tasa):,} de {celdas_con_dato:,} celdas con accidentes alcanzan "
            f"el piso de {PISO}; el resto se suprime", pad=10)
barra = fig.colorbar(hb, ax=ax, fraction=0.03, pad=0.02, extend="max")
barra.set_label("% de accidentes con víctimas", size=8.5)
barra.outline.set_visible(False)
plt.show()

print(f"Promedio ZMM: {100*d.hay_victimas.mean():.1f} % de accidentes con víctimas")
```

Proporción de accidentes con víctimas por celda de 500 m
1,976 de 4,430 celdas con accidentes alcanzan el piso de 30; el resto



Promedio ZMM: 5.7 % de accidentes con víctimas

El patrón se **invierte** respecto al § 3, y no es una impresión del mapa:

```
centros_t = hb.get_offsets()  
conteo_t = np.asarray(ax.hexbin(acc.x, acc.y, gridsize=malla, extent=EXT,
```

```

mincnt=PISO, visible=False).get_array())
distancia = np.hypot(centros_t[:, 0] - px.mean(), centros_t[:, 1] - py.mean()) / 1000

cuartil = pd.qcut(distancia, 4, labels=["1º (centro)", "2º", "3º", "4º (periferia)"])
comparacion = (pd.DataFrame({"% con víctimas": tasa, "accidentes por celda": conteo_t})
               .groupby(cuartil, observed=True).mean().round(1))
comparacion.index.name = "cuartil de distancia al centroide"
print(comparacion.to_string())
print(f"\nSpearman conteo vs tasa de víctimas:      {pd.Series(conteo_t).corr(pd.Series(tasa), metf
print(f"Spearman distancia al centro vs tasa:      {pd.Series(distancia).corr(pd.Series(tasa), metf

```

	% con víctimas	accidentes por celda
1º (centro)	5.3	231.2
2º	5.4	177.6
3º	7.6	131.4
4º (periferia)	9.1	88.5

Spearman conteo vs tasa de víctimas: -0.246
 Spearman distancia al centro vs tasa: +0.271

Del centro a la periferia la proporción con víctimas sube de 5.3 % a 9.8 % mientras el volumen por celda cae de 327 a 80 accidentes, y la correlación de rangos entre conteo y tasa es **negativa** (-0.24). Los cruces saturados producen muchos accidentes de baja energía; los tramos periféricos, pocos y graves. Un ranking por conteo y un ranking por tasa señalan ciudades distintas, y las dos lecturas son ciertas — lo que no se puede es leer el § 3 como un mapa de gravedad.

Los fallecimientos no admiten este tratamiento. 1,075 muertos en seis años, repartidos en 982 nodos, con un máximo de 4 en un solo nodo. No hay masa para una tasa de mortalidad por celda ni para un «cruce más letal»: cualquier ranking así sería ruido ordenado.

9. Gi* de Getis-Ord: del «se ve caliente» al «lo está, con *p*»

Un mapa de conteos no distingue un aglomerado real de la variación que produciría cualquier reparto aleatorio. El Gi* de Getis-Ord compara la suma en la vecindad de cada celda contra la que se esperaría si los conteos estuvieran repartidos al azar entre celdas, y devuelve un puntaje *z*.

Se implementa a mano: `libpysal`/`esda` no están en las dependencias del proyecto y la estadística son seis líneas.

```

fig_tmp, ax_tmp = plt.subplots()
base = ax_tmp.hexbin(acc.x, acc.y, gridsize=malla, extent=EXT, mincnt=1)
centros, conteos = base.get_offsets(), base.get_array().astype(float)
plt.close(fig_tmp)

```

```

def gi_estrella(centros, y, banda):
    """Gi* de Getis-Ord con banda de distancia fija. Devuelve (z, p a dos colas).

    La diagonal de W vale 1: es la variante con estrella, que incluye a la
    propia celda en su vecindad.
    """
    n = len(y)
    media = y.mean()
    s = np.sqrt((y ** 2).sum() / n - media ** 2)
    D = np.sqrt(((centros[:, None, :] - centros[None, :, :]) ** 2).sum(-1))
    W = (D <= banda).astype(float)
    Wi, Si = W.sum(1), (W ** 2).sum(1)
    z = (W @ y - media * Wi) / (s * np.sqrt((n * Si - Wi ** 2) / (n - 1)))
    return z, 2 * stats.norm.sf(np.abs(z))

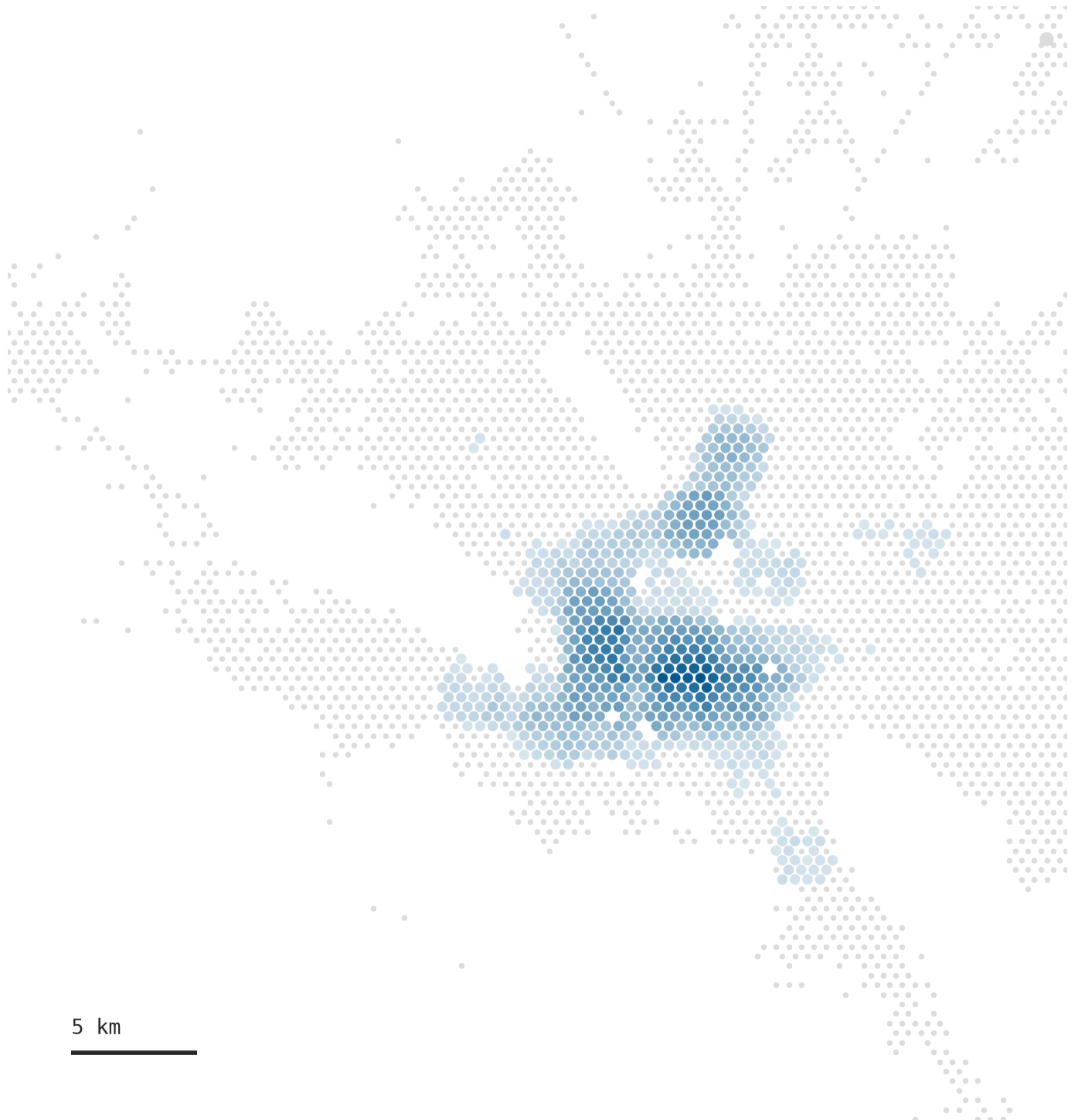
BANDA = 1500.0
z, p = gi_estrella(centros, conteos, BANDA)
significativa = multipletests(p, alpha=0.05, method="fdr_bh")[0]

fig, ax = plt.subplots(figsize=(8.6, 8.2))
lienzo_mapa(ax, EXT, escala_km=5)
ax.scatter(centros[~significativa, 0], centros[~significativa, 1], s=4,
           c=FONDO_PTS, linewidths=0, zorder=2, label="sin significancia")
orden = np.argsort(z[significativa])
sc = ax.scatter(centros[significativa][orden, 0], centros[significativa][orden, 1],
               c=z[significativa][orden], s=14, cmap=SECUENCIAL, linewidths=0,
               vmin=0, zorder=4)
ax.set_title(f"Gi* de Getis-Ord sobre celdas de {LADO:.0f} m, banda de "
             f"{BANDA/1000:.1f} km\n{significativa.sum():,} de {len(z):,} celdas "
             f"significativas tras corrección FDR (Benjamini-Hochberg)", pad=10)
ax.legend(loc="upper right", markerscale=2.5)
barra = fig.colorbar(sc, ax=ax, fraction=0.03, pad=0.02)
barra.set_label("puntaje z de Gi*", size=8.5)
barra.outline.set_visible(False)
plt.show()

print(f"z: mínimo {z.min():.1f}, máximo {z.max():.1f}")
print(f"Celdas con z > 2.58 (p < 0.01): {(z > 2.58).sum():,}")
print(f"Celdas con z < -2.58 (agrupamiento frío): {(z < -2.58).sum():,}")

```

Gi* de Getis-Ord sobre celdas de 500 m, banda de 1.5 km
653 de 4,430 celdas significativas tras corrección FDR (Benjamini-Ho



z: mínimo -2.4, máximo 18.0

Celdas con $z > 2.58$ ($p < 0.01$): 675

Celdas con $z < -2.58$ (agrupamiento frío): 0

Dos decisiones que hay que declarar porque cambian el resultado:

- **El universo son las celdas con al menos un accidente**, no el rectángulo completo. Meter el cerro de la Silla y el semidesierto del norte como celdas de valor cero infla el z de toda la mancha urbana contra un vacío que no es parte de la red viaria. Así, el universo es «la ciudad» y la pregunta es dónde se agrupa *dentro* de ella.
- **No hay agrupamiento frío** (z mínimo ≈ -2.3): no existen zonas con significativamente menos accidentes que sus vecinas, solo zonas sin accidentes, que quedan fuera del universo. Por eso la escala es **secuencial y no divergente**: no hay polaridad que representar, y una divergente sugeriría un polo frío inexistente.

10. DBSCAN: la versión que falla y la que sirve

DBSCAN sobre ubicaciones de accidentes es el reflejo obvio y, tal cual, **no funciona**. En una conurbación los accidentes forman una mancha continua: con cualquier **eps** que agrupe cruces vecinos, el algoritmo encadena la ciudad entera en un solo grupo. Vale la pena mostrarlo en lugar de esconderlo.

```
X = nodos[["x", "y"]].to_numpy()
peso = nodos.n.to_numpy()

etiqa_mal = DBSCAN(eps=300, min_samples=100).fit_predict(X, sample_weight=peso)
tam = (pd.Series(peso).groupby(etiq_mal).sum().drop(-1, errors="ignore")
      .sort_values(ascending=False))
print(f"DBSCAN ponderado, eps=300 m: {len(tam)} grupos")
print(f"  el mayor se come {tam.iloc[0]:,} accidentes = {100*tam.iloc[0]/N:.0f} % del total")
print(f"  el segundo:      {tam.iloc[1]:,} accidentes")
```

```
DBSCAN ponderado, eps=300 m: 165 grupos
  el mayor se come 276,998 accidentes = 73 % del total
  el segundo:      4,555 accidentes
```

El grupo mayor absorbe el 73 % de los accidentes: es el contorno de la conurbación, no una zona de riesgo. El arreglo no es mover **eps**, es **cambiar el universo**: agrupar solo los cruces que ya son puntos negros. Entonces los grupos son corredores de puntos negros, que es el objeto que el § 3 deja ver.

```
UMBRAL = 80          # accidentes en seis años, ~13 por año
pn2 = nodos[nodos.n >= UMBRAL].copy()
pn2["corredor"] = DBSCAN(eps=300, min_samples=3).fit_predict(pn2[["x", "y"]].to_numpy())

corr = (pn2[pn2.corredor >= 0].groupby("corredor")
      .agg(accidentes=("n", "sum"), cruces=("n", "size"),
          x=("x", "mean"), y=("y", "mean"),
          municipio=("NOM_MUN", lambda s: s.mode().iat[0]))
      .sort_values("accidentes", ascending=False))

# El nombre del corredor es la vialidad más repetida entre sus cruces contando
# las dos columnas de calle: con solo `calle1_norm` el eje que define al grupo
# se pierde cada vez que quedó capturado como segunda vialidad.
```

```

def via_dominante(cid):
    pts = pn2.index[pn2.corredor == cid]
    vias = pd.concat([pn2.calle1_norm[pts], pn2.calle2_norm[pts]])
    return vias.value_counts().index[0]

corr["via"] = [via_dominante(cid) for cid in corr.index]

fig, ejes = plt.subplots(1, 2, figsize=(12.6, 6.6))

lienzo_mapa(ejes[0], EXT, escala_km=10, fondo=FONDO)
mayor = tam.index[0]
es_mayor = etiq_mal == mayor
ejes[0].scatter(nodos.x[~es_mayor], nodos.y[~es_mayor], s=2, c=FONDO_PTS,
                linewidths=0, zorder=3)
ejes[0].scatter(nodos.x[es_mayor], nodos.y[es_mayor], s=2, c=ROJO, alpha=0.5,
                linewidths=0, zorder=4)
ejes[0].set_title(f"No sirve: DBSCAN ponderado sobre los 54,742 cruces\n"
                  f"un grupo encadena {100*tam.iloc[0]/N:.0f} % de los accidentes",
                  fontsize=10, color=GRAFITO, loc="left", pad=8)

lienzo_mapa(ejes[1], EXT, escala_km=10, fondo=FONDO)
for cid, r in corr.iterrows():
    pts = pn2[pn2.corredor == cid]
    casco = MultiPoint(list(zip(pts.x, pts.y))).convex_hull.buffer(220)
    xs, ys = casco.exterior.xy
    ejes[1].fill(xs, ys, facecolor=AZUL, alpha=0.35, edgecolor=AZUL,
                linewidth=1.1, zorder=4)
insignias(ejes[1], corr.x[:10], corr.y[:10])
ejes[1].set_title(f"Sirve: DBSCAN sobre los {len(pn2):,} cruces con  $\geq$  {UMBRALE} accidentes\n"
                  f"{len(corr)} corredores, el mayor con {corr.accidentes.iloc[0]:,} accidentes",
                  fontsize=10, color=GRAFITO, loc="left", pad=8)

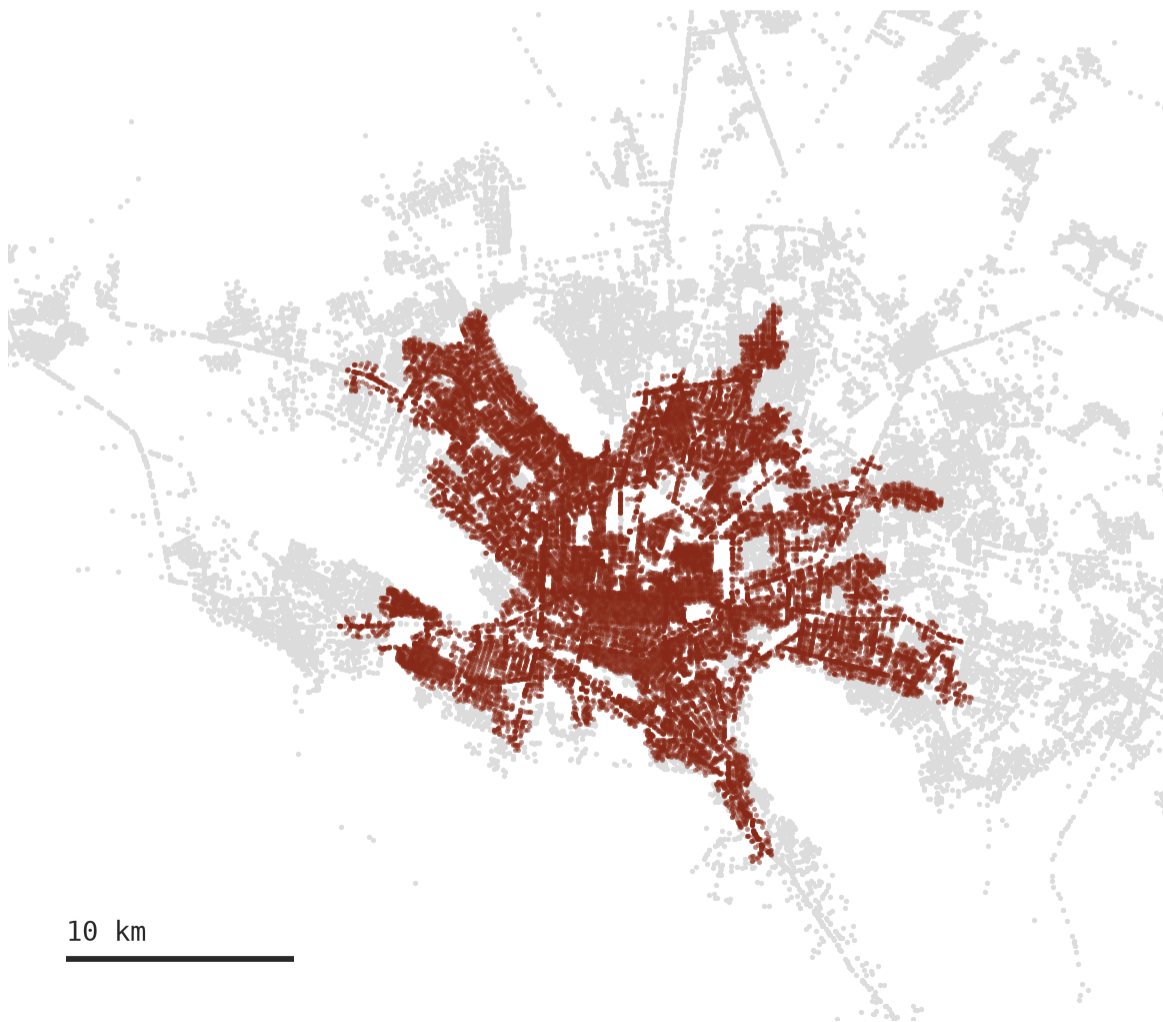
fig.suptitle("El mismo algoritmo, dos universos", color=ROJO, size=11.5,
             weight="bold", x=0.062, ha="left", y=0.98)
plt.show()

ranking = corr.head(10).reset_index(drop=True)
ranking.index += 1
print(ranking[["via", "municipio", "cruces", "accidentes"]].
      .rename(columns={"via": "vialidad dominante"}).to_string())

```

El mismo algoritmo, dos universos

No sirve: DBSCAN ponderado sobre los 54,742 cruces
un grupo encadena 73 % de los accidentes



	vialidad dominante	municipio	cruces	accidentes
1	CUAUHTEMOC	Monterrey	32	4842
2	UNIVERSIDAD	San Nicolás de los Garza	22	4581
3	DOCTOR JOSE ELEUTERIO GONZALEZ	Monterrey	16	3535
4	FRANCISCO I. MADERO	Monterrey	24	3422
5	MANUEL L. BARRAGAN	San Nicolás de los Garza	9	2993
6	DOCTOR JOSE ELEUTERIO GONZALEZ	Monterrey	11	2549
7	CALZADA DEL VALLE	San Pedro Garza García	14	2331
8	UNIVERSIDAD	San Nicolás de los Garza	9	2324
9	DOCTOR JOSE ELEUTERIO GONZALEZ	Monterrey	9	2287
10	DOCTOR IGNACIO MORONES PRIETO	Monterrey	7	1928

Qué aporta DBSCAN sobre el § 3. El mapa de burbujas ordena cruces; esto ordena **corredores**, que es la unidad en que se interviene una avenida. Un corredor de 9 cruces con 4,800 accidentes pesa más que el cruce individual más alto, y en el mapa de burbujas se lee como nueve puntos medianos.

11. Tipología de cruces

Lo que ni el conteo ni la densidad contestan: **dos cruces con el mismo número de accidentes pueden ser problemas distintos**. Uno acumula alcances a la hora pico; otro atropella peatones; otro concentra choques contra objeto fijo de madrugada. Cada uno pide una intervención diferente, y esa diferencia está en el perfil, no en el volumen.

Universo: los cruces con al menos 30 accidentes en seis años — 2,302 cruces, 54.6 % de todos los registros. Por debajo de eso un porcentaje sobre 8 accidentes no es un perfil.

```
MINIMO = 30
sel = d[d.id_punto_4d.isin(nodos.index[nodos.n >= MINIMO]).copy()]
sel["madrugada"] = sel.hora.between(0, 5)
sel["finde"] = sel.dia_semana.isin(["Sábado", "Domingo"])
sel["moto"] = sel.TIPACCID == 10
sel["vulnerable"] = sel.TIPACCID.isin([2, 11]) # peatón o ciclista
sel["objeto_fijo"] = sel.TIPACCID == 4 # un solo vehículo, pérdida de control

gs = sel.groupby("id_punto_4d", observed=True)
RASGOS = ["madrugada", "finde", "moto", "vulnerable", "objeto_fijo",
          "victimas", "vehiculos"]
P = pd.DataFrame({
    "n": gs.size(),
    "madrugada": gs.madrugada.mean(),
    "finde": gs.finde.mean(),
    "moto": gs.moto.mean(),
    "vulnerable": gs.vulnerable.mean(),
    "objeto_fijo": gs.objeto_fijo.mean(),
    "victimas": gs.hay_victimas.mean(),
    "vehiculos": gs.n_vehiculos.mean(),
})
Z = StandardScaler().fit_transform(P[RASGOS])
print(f"{len(P):,} cruces × {len(RASGOS)} rasgos | "
      f"{P.n.sum():,} accidentes = {100*P.n.sum()/N:.1f} % de la base")
```

2,302 cruces × 7 rasgos | 207,207 accidentes = 54.6 % de la base

El volumen **no** entra como rasgo: si entrara, *k*-medias separaría cruces grandes de chicos, que es lo que ya hace el § 3. Los rasgos son todos composicionales, así que los grupos son tipos de problema, no tamaños.

Cuántos tipos

```

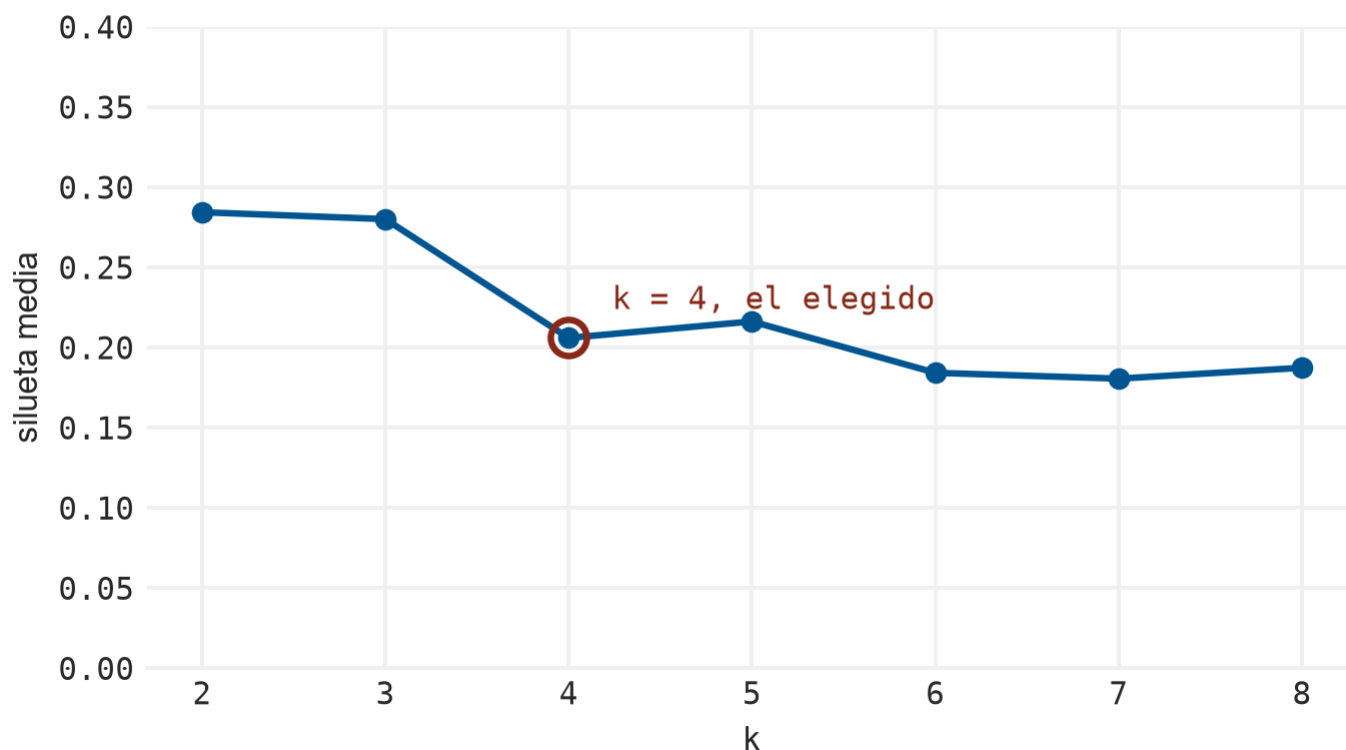
ks = range(2, 9)
siluetas = [silhouette_score(Z, KMeans(k, n_init=25, random_state=0).fit_predict(Z))
             for k in ks]

fig, ax = plt.subplots(figsize=(6.0, 3.2))
ax.plot(list(ks), siluetas, color=AZUL, marker="o", ms=5)
ax.plot(4, siluetas[2], "o", ms=10, mfc="none", mec=ROJO, mew=1.8)
ax.annotate("k = 4, el elegido", (4, siluetas[2]), textcoords="offset points",
           xytext=(12, 8), fontsize=8.5, color=ROJO, family="monospace")
ax.set_title("Silueta media según el número de tipos")
ax.set_xlabel("k")
ax.set_ylabel("silueta media")
ax.set_ylim(0, 0.4)
pulis(ax)
plt.show()

for k, s in zip(ks, siluetas):
    print(f" k={k}  silueta {s:.3f}")

```

Silueta media según el número de tipos



```

k=2  silueta 0.284
k=3  silueta 0.280
k=4  silueta 0.206
k=5  silueta 0.216
k=6  silueta 0.184
k=7  silueta 0.181
k=8  silueta 0.187

```

La silueta es baja y hay que decirlo. El máximo está en $k = 2$ con 0.28 y $k = 4$ da 0.21. Con esos valores **no hay grupos naturales bien separados**: el perfil de un cruce es continuo y la partición es una discretización útil, no un descubrimiento de categorías latentes. Se elige $k = 4$ porque $k = 2$ solo separa «con víctimas / sin víctimas» —que ya es una columna— y porque los cuatro centroides de $k = 4$ son interpretables y estables entre semillas. Si este documento reportara $k = 4$ como «los cuatro tipos de cruce de Monterrey» estaría sobrevendiendo una silueta de 0.21.

Los cuatro perfiles

```
K = 4
km = KMeans(K, n_init=25, random_state=0).fit(Z)
P["grupo"] = km.labels_

perfiles = P.groupby("grupo")[RASGOS].mean()

# El índice de k-medias depende de la semilla; el nombre se deriva del centroide
# para que la etiqueta no dependa de un número arbitrario.
NOMBRES = {}
NOMBRES[perfiles.objeto_fijo.idxmax()] = "Pérdida de control nocturna"
NOMBRES[perfiles.vulnerable.idxmax()] = "Exposición de peatón y ciclista"
NOMBRES[perfiles.moto.drop(index=list(NOMBRES)).idxmax()] = "Corredor de motocicleta"
NOMBRES[(set(perfiles.index) - set(NOMBRES)).pop()] = "Alto flujo, daños materiales"
P["tipo"] = P.grupo.map(NOMBRES)

centroides = (pd.DataFrame(km.cluster_centers_, columns=RASGOS)
               .rename(index=NOMBRES)
               .loc[[NOMBRES[i] for i in perfiles.index]])

fig, ax = plt.subplots(figsize=(8.0, 3.4))
lim = np.abs(km.cluster_centers_).max()
im = ax.imshow(centroides.to_numpy(), cmap=DIVERGENTE, vmin=-lim, vmax=lim,
               aspect="auto")
ETIQUETA = {"madrugada": "% madrugada", "finde": "% fin de semana",
            "moto": "% motocicleta", "vulnerable": "% peatón/ciclista",
            "objeto_fijo": "% objeto fijo", "victimas": "% con víctimas",
            "vehiculos": "vehículos por accidente"}
ax.set_xticks(range(len(RASGOS)), [ETIQUETA[r] for r in RASGOS],
              rotation=30, ha="right")
ax.set_yticks(range(K), centroides.index)
ax.grid(False)
for lado in ax.spines.values():
    lado.set_visible(False)
ax.tick_params(length=0)
for i in range(K):
    for j in range(len(RASGOS)):
        v = centroides.iat[i, j]
        ax.text(j, i, f"{v:+.1f}", ha="center", va="center", fontsize=7.5,
               family="monospace", weight="bold",
```

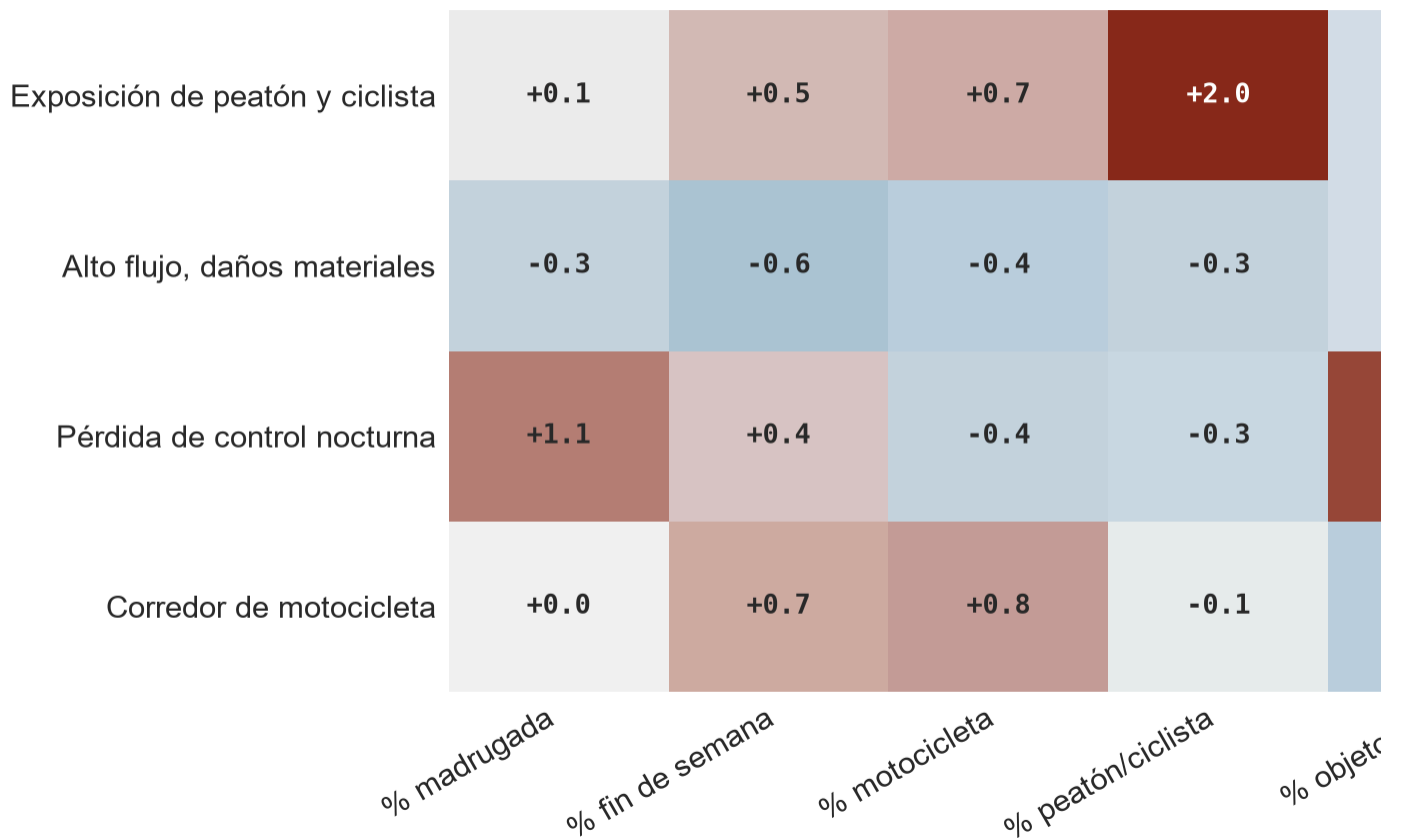
```

        color=BLANCO if abs(v) > 0.6 * lim else GRAFITO)
ax.set_title("Perfil de cada tipo, en desviaciones estándar respecto al cruce promedio")
barra = fig.colorbar(im, ax=ax, fraction=0.025, pad=0.02)
barra.set_label("desv. est.", size=8.5)
barra.outline.set_visible(False)
plt.show()

tabla = pd.DataFrame({
    "cruces": P.tipo.value_counts(),
    "accidentes": P.groupby("tipo").n.sum(),
    "accid./cruce": P.groupby("tipo").n.mean().round(0),
})
for r in RASGOS[:-1]:
    tabla[r] = (100 * P.groupby("tipo")[r].mean()).round(1)
tabla["vehículos"] = P.groupby("tipo").vehiculos.mean().round(2)
print(tabla.loc[centroides.index].to_string())

```

Perfil de cada tipo, en desviaciones estándar respecto al cruce promedio



	cruces	accidentes	accid./cruce	mañana	finde	moto
vulnerable	objeto_fijo	victimas	vehículos			
Exposición de peatón y ciclista	276	17135	62.0	8.1	27.2	8.9
7.7	8.3	13.2	1.88			
Alto flujo, daños materiales	1163	123869	107.0	5.7	17.3	3.6

1.3	8.4	2.8	1.97					
Pérdida de control nocturna			338	29471	87.0	14.4	26.2	3.9
1.4	25.4	4.9	1.77					
Corredor de motocicleta			525	36732	70.0	7.8	28.7	9.6
2.0	6.6	5.5	1.95					

Esta es la única escala **divergente** del documento, y le corresponde: el dato es la distancia al promedio, tiene signo, y el gris del centro es el cruce promedio. Azul por debajo, rojo por encima.

Dónde está cada tipo

Cuatro tipos no se pueden pintar con cuatro tonos en un mapa: bajo daltonismo los pares dejan de separarse y un mapa de dispersión no admite etiquetado directo. La regla del manual aplica sin excepción — **más de dos series, paneles pequeños**. Cada panel un tipo, un solo tono, el mismo encuadre.

```
P = P.join(nodos[["x", "y", "NOM_MUN", "calle1_norm", "calle2_norm"]])

fig, ejes = plt.subplots(2, 2, figsize=(11.0, 10.4))
for ax, tipo in zip(ejes.ravel(), centroides.index):
    sub = P[P.tipo == tipo]
    lienzo_mapa(ax, EXT, escala_km=10, fondo=FONDO)
    ax.scatter(sub.x, sub.y, s=sub.n / 7, c=AZUL, alpha=0.6, edgecolors=BLANCO,
               linewidths=0.4, zorder=4)
    ax.set_title(f"{tipo}\n{len(sub):,} cruces · {sub.n.sum():,} accidentes",
               fontsize=10, color=GRAFITO, loc="left", pad=6)

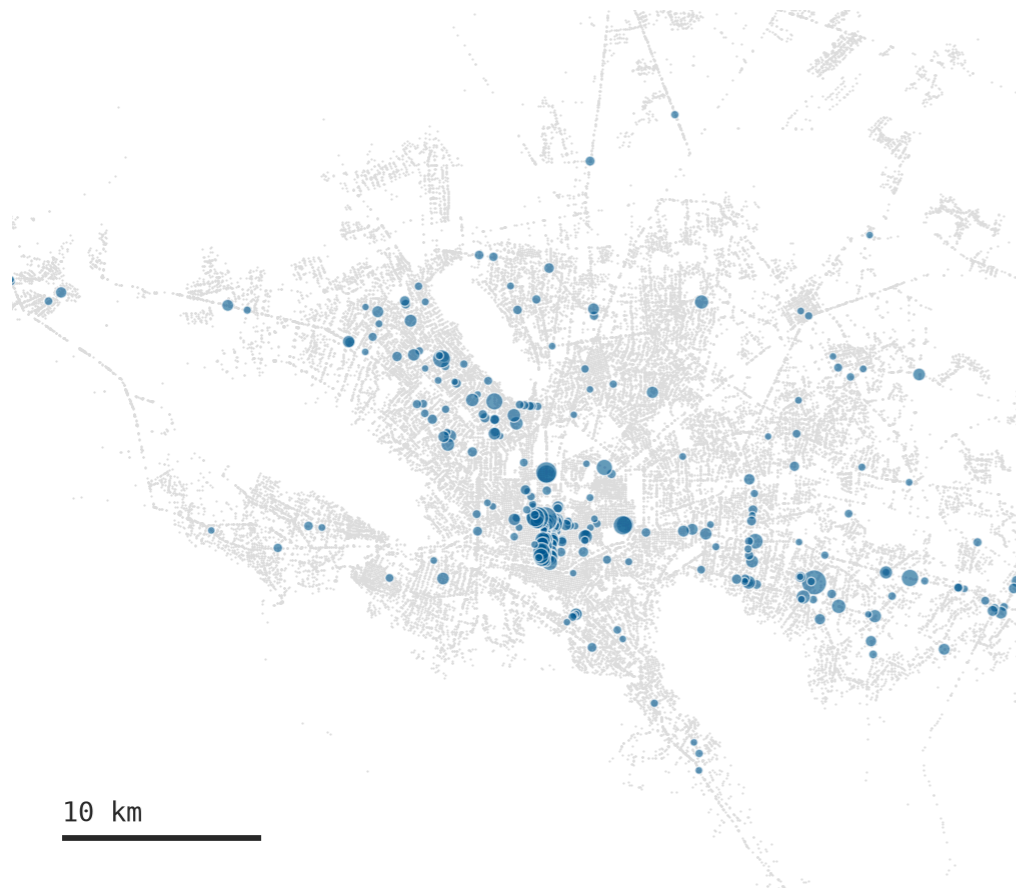
fig.suptitle("Tipología de cruces, un panel por tipo "
            "(área proporcional a los accidentes del cruce)",
            color=ROJO, size=11.5, weight="bold", x=0.065, ha="left", y=0.965)
plt.show()

P["dist_centro"] = np.hypot(P.x - px.mean(), P.y - py.mean()) / 1000
print("Distancia al centroide de los accidentes, en km:")
print(P.groupby("tipo").dist_centro.agg(["mean", "median"]).round(1)
      .loc[centroides.index].to_string())

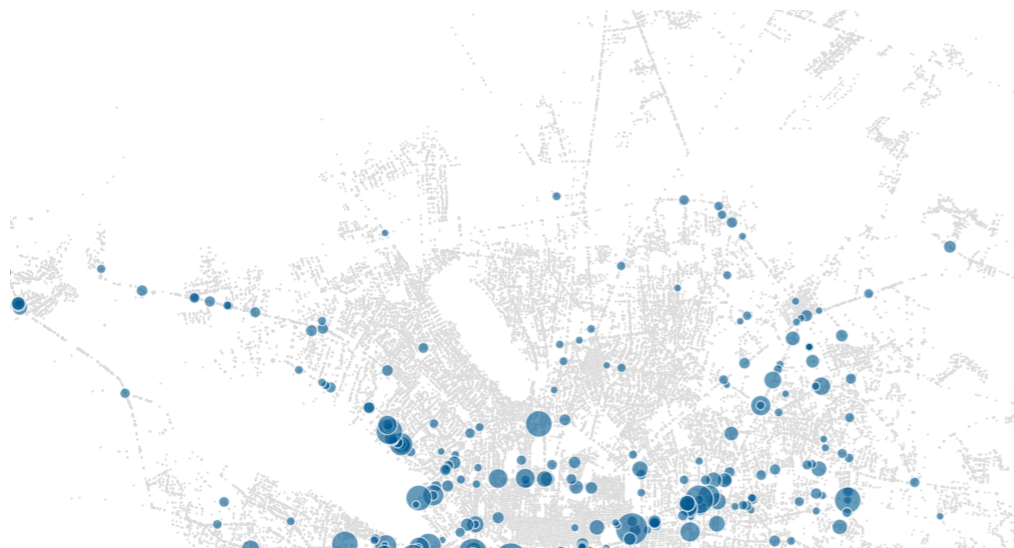
comp = (pd.crosstab(P.NOM_MUN, P.tipo, normalize="index").mul(100).round(1)
        [list(centroides.index)])
print(f"\nComposición por municipio, en % de los cruces del municipio "
      f"({len(comp)} de 18 municipios tienen al menos un cruce con "
      f"{MINIMO}+ accidentes):")
print(comp.sort_values(centroides.index[1], ascending=False).to_string())
```

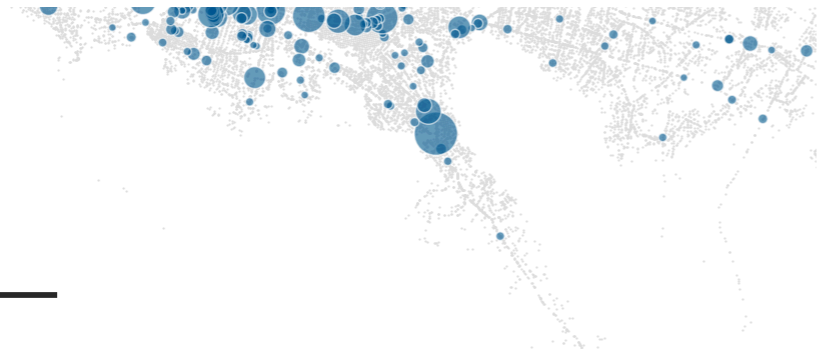
Tipología de cruces, un panel por tipo (área proporcional a los accid

Exposición de peatón y ciclista
276 cruces · 17,135 accidentes

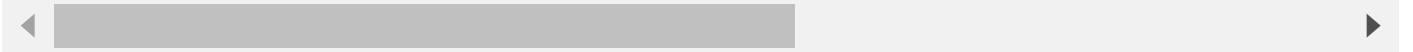


Pérdida de control nocturna
338 cruces · 29,471 accidentes





10 km



Distancia al centroide de los accidentes, en km:

	mean	median
Exposición de peatón y ciclista	8.6	6.7
Alto flujo, daños materiales	6.7	5.8
Pérdida de control nocturna	10.6	8.8
Corredor de motocicleta	9.8	8.9

Composición por municipio, en % de los cruces del municipio (16 de 18 municipios tienen al menos un cruce con 30+ accidentes):

tipo Exposición de peatón y ciclista Alto flujo, daños materiales Pérdida de control nocturna Corredor de motocicleta

NOM_MUN

San Nicolás de los Garza	2.1	86.9
2.7	8.2	
San Pedro Garza García	1.2	70.0
16.7	12.1	
Santa Catarina	11.8	52.9
17.6	17.6	
Monterrey	14.7	50.3
11.3	23.7	
Apodaca	10.3	29.9
37.4	22.4	
General Escobedo	9.9	29.7
5.9	54.5	
García	15.8	21.1
38.6	24.6	
Pesquería	0.0	20.0
20.0	60.0	
Guadalupe	23.2	11.3
32.8	32.8	
Ciénega de Flores	0.0	9.1
0.0	90.9	
Cadereyta Jiménez	0.0	0.0
75.0	25.0	
El Carmen	0.0	0.0
0.0	100.0	
General Zuazua	0.0	0.0
0.0	100.0	
Juárez	57.1	0.0

37.1	5.7		
Salinas Victoria		50.0	0.0
0.0	50.0		
Santiago		0.0	0.0
83.3	16.7		

Los tipos tienen **geografía**, y la distancia al centroide lo mide: el alto flujo con daños materiales ocupa el primer anillo (6.7 km de media), mientras la pérdida de control nocturna se va a la periferia y a los accesos carreteros (10.6 km). El corredor de motocicleta queda en medio, extendido hacia el norte —Escobedo tiene el 54.5 % de sus cruces en ese tipo y Ciénega de Flores el 90.9 %—, y la exposición de peatón y ciclista se concentra donde la red mezcla tránsito rápido con actividad a pie: Juárez, 57.1 % de sus cruces.

La partición depende de la semilla mucho menos de lo que sugiere una silueta de 0.21, porque lo inestable son los cruces de frontera y no los centroides:

```
for semilla in (1, 7, 42):
    otra = KMeans(K, n_init=25, random_state=semilla).fit(Z)
    perf = pd.DataFrame(otra.labels_, index=P.index, columns=["g"]).join(P[RASGOS]) \
        .groupby("g")[RASGOS].mean()
    nombres = {}
    nombres[perf.objeto_fijo.idxmax()] = "Pérdida de control nocturna"
    nombres[perf.vulnerable.idxmax()] = "Exposición de peatón y ciclista"
    nombres[perf.moto.drop(index=list(nombres)).idxmax()] = "Corredor de motocicleta"
    nombres[(set(perf.index) - set(nombres)).pop()] = "Alto flujo, daños materiales"
    coincide = 100 * (pd.Series(otra.labels_, index=P.index).map(nombres) == P.tipo).mean()
    print(f"semilla {semilla:>2}: {coincide:.1f} % de los cruces conserva su tipo")
```

```
semilla 1: 99.8 % de los cruces conserva su tipo
semilla 7: 99.6 % de los cruces conserva su tipo
semilla 42: 99.8 % de los cruces conserva su tipo
```

12. Los puntos negros, con su tipo

El producto final: el ranking del § 3 con la lectura del § 11 pegada. La columna que decide la intervención no es el conteo, es el tipo.

```
def corto(s, ancho=26):
    """Recorta un nombre de vialidad para que la tabla quepa.

    `Calle` y `Avenida` son ruido de captura, no parte del nombre: aparecen en
    unos registros y en otros no, para la misma vialidad.
    """
    s = str(s).title()
    for prefijo in ("Calle ", "Avenida ", "Ave "):
        while s.startswith(prefijo):
            s = s[len(prefijo):]
    s = (s.replace("Licenciado ", "Lic. ").replace("Doctor ", "Dr. ")
        .replace("General ", "Gral. ").replace("Carretera ", "Carr. "))
```

```

return s if len(s) <= ancho else s[:ancho - 1] + "..."

tabla_final = P.assign(
    cruce=[f"{corto(a)} x {corto(b)}" for a, b in zip(P.calle1_norm, P.calle2_norm)],
    **{"% víct.": (100 * P.victimas).round(1),
       "% moto": (100 * P.moto).round(1),
       "% madr.": (100 * P.madrugada).round(1)},
).rename(columns={"n": "accid.", "NOM_MUN": "municipio"})

COLS = ["cruce", "municipio", "accid.", "% víct.", "% moto", "% madr.", "tipo"]
print("Los 15 cruces con más accidentes de la ZMM, 2019-2024\n")
print(tabla_final.sort_values("accid.", ascending=False).head(15)[COLS]
      .to_string(index=False))

```

Los 15 cruces con más accidentes de la ZMM, 2019-2024

		cruce	municipio	accid.	% víct.
% moto	% madr.	tipo			
		Fray Luis De Leon x Fidel Velazquez	San Nicolás de los Garza	1024	2.1
3.7	3.4	Alto flujo, daños materiales			
		Eugenio Garza Sada x Lazaro Cardenas	Monterrey	1004	2.6
2.1	7.6	Pérdida de control nocturna			
		Acapulco x Carr. Miguel Aleman	San Nicolás de los Garza	986	1.8
3.4	3.7	Alto flujo, daños materiales			
		Manuel L. Barragan x Manuel L. Barragan	San Nicolás de los Garza	917	2.9
3.6	5.8	Alto flujo, daños materiales			
		Simon Bolivar x Paseo De Los Leones	Monterrey	810	1.5
4.6	4.0	Alto flujo, daños materiales			
		Felix Uresti Gomez x Constitucion	Monterrey	761	2.6
2.0	8.9	Alto flujo, daños materiales			
		San Jeronimo x Dr. Jose Eleuterio Gonzal...	Monterrey	706	0.7
2.1	6.2	Alto flujo, daños materiales			
		Paseo De Los Leones x Pedro Infante	Monterrey	698	2.1
3.9	3.7	Alto flujo, daños materiales			
		San Pedro x Calzada Del Valle	San Pedro Garza García	698	0.9
2.4	10.2	Alto flujo, daños materiales			
		Revolucion x Chapultepec	Monterrey	693	3.3
2.2	6.1	Alto flujo, daños materiales			
		Dr. Jose Eleuterio Gonzal... x Terranova	Monterrey	671	0.9
4.2	2.4	Alto flujo, daños materiales			
		Gral. Bernardo Reyes x Fidel Velazquez	Monterrey	670	3.0
4.0	5.7	Alto flujo, daños materiales			
		Constitucion x Fundidora	Monterrey	624	2.7
2.2	4.3	Alto flujo, daños materiales			
		Eugenio Garza Sada x Salto Del Agua	Monterrey	595	1.2
2.0	3.9	Alto flujo, daños materiales			
		Gral. Pablo Gonzalez Garza x Dr. Jose Eleuterio Gonzal...	Monterrey	593	1.7
4.2	4.4	Alto flujo, daños materiales			

Un ranking por volumen está dominado por un solo tipo: **13 de los 15 son «alto flujo, daños materiales»**, que es tautológico —el tipo se definió, entre otras cosas, por tener muchos accidentes leves—. El ranking útil es **dentro de cada tipo**, porque ahí cada fila compite contra cruces que comparten el problema.

```

for tipo in centroides.index:
    print(f"\n{tipo.upper()}")
    sub = tabla_final[tabla_final.tipo == tipo].sort_values("accid.", ascending=False)
    print(sub.head(6)[COLS[:-1]].to_string(index=False))

```

EXPOSICIÓN DE PEATÓN Y CICLISTA

	cruce	municipio	accid.	% víct.	% moto	% madr.
Pablo Livas × Maestro Israel Cavazos Ga...	Guadalupe		335	12.5	9.0	15.2
Jose Maria Pino Suarez × Cristobal Colon	Monterrey		314	11.5	2.9	5.7
Cristobal Colon × Julian Villagran	Monterrey		271	9.6	5.9	4.4
Manuel L. Barragan × Martin Carrera	Monterrey		243	15.6	14.4	4.9
Cristobal Colon × Francisco I. Madero	Monterrey		202	12.9	2.5	5.0
Manuel L. Barragan × Jose Mariano Salas	Monterrey		193	22.8	9.3	4.1

ALTO FLUJO, DAÑOS MATERIALES

	cruce	municipio	accid.	% víct.	% moto	%
3.4	Fray Luis De Leon × Fidel Velazquez	San Nicolás de los Garza	1024	2.1	3.7	
3.7	Acapulco × Carr. Miguel Aleman	San Nicolás de los Garza	986	1.8	3.4	
5.8	Manuel L. Barragan × Manuel L. Barragan	San Nicolás de los Garza	917	2.9	3.6	
4.0	Simon Bolivar × Paseo De Los Leones	Monterrey	810	1.5	4.6	
8.9	Felix Uresti Gomez × Constitucion	Monterrey	761	2.6	2.0	
6.2	San Jeronimo × Dr. Jose Eleuterio Gonzal...	Monterrey	706	0.7	2.1	

PÉRDIDA DE CONTROL NOCTURNA

	cruce	municipio	accid.	% víct.	% moto	% madr.
Eugenio Garza Sada × Lazaro Cardenas	Monterrey		1004	2.6	2.1	7.6
Venustiano Carranza × Constitucion	Monterrey		570	2.5	1.8	6.5
Constitucion × Cristobal Colon	Monterrey		560	5.4	3.0	10.4
Eugenio Garza Sada × Dr. Ignacio Morones Prieto	Monterrey		548	3.3	2.6	9.7
Dr. Ignacio Morones Prieto × Dr. S. Sepulveda	Monterrey		474	1.7	1.9	8.2
Gral. Bonifacio Salinas L... × Carr. A Miguel Aleman	Guadalupe		441	5.9	5.0	10.7

CORREDOR DE MOTOCICLETA

	cruce	municipio	accid.	% víct.	% moto	% madr.
Luis Donaldo Colosio Murr... × Cabezada	Monterrey		398	5.0	5.0	4.3
Puerta De Hierro × Abraham Lincoln	Monterrey		396	2.5	7.6	7.3
Felix Uresti Gomez × Cristobal Colon	Monterrey		375	6.7	6.1	4.8
Aztlan × Lic. Raul Rangel Frias	Monterrey		373	4.3	9.7	8.6
Abraham Lincoln × De Los Astros	Monterrey		334	7.5	4.8	4.5
Felix Uresti Gomez × Francisco I. Madero	Monterrey		332	5.7	5.4	10.2

Caso frontera, para no leer la tabla como un veredicto. Garza Sada × Lázaro Cárdenas, el segundo cruce con más accidentes de la ZMM, cae en «pérdida de control nocturna» con apenas 7.6 % de madrugada. Con silueta 0.21 eso es exactamente lo que se advirtió en el § 11: los cruces que quedan entre dos centroides se asignan a uno de ellos y la etiqueta individual no es evidencia. Los agregados por tipo son estables; la fila suelta, no.

Limitaciones

Lo que este documento **no** puede afirmar, con el dato que hay:

Límite	Por qué
No hay tasa de riesgo, solo conteos	Falta el denominador de exposición: aforo vehicular, longitud de vialidad, población. Un cruce con más accidentes puede ser solo un cruce con más coches. Sin eso, cualquier modelo de conteo (Poisson, binomial negativa) predice «dónde hay tráfico» y no «dónde hay riesgo». Por eso este documento no ajusta ninguno
No hay contornos ni mapa base	<code>data/raw/</code> no trae el marco geoestadístico del INEGI. Falta el polígono municipal y el AGEB para unir por <code>CVE_MUN</code> , y sin población de 2020 no hay accidentes por habitante ni coropletas
No se comparan municipios en los campos del conductor	La tasa de captura de <code>ALIENTO</code> , <code>EDAD</code> y <code>CINTURON</code> va de 10.4 % en San Pedro a 100 % en Monterrey (<code>limpieza.tasa_captura</code>). Un mapa de «% con aliento alcohólico» sería un mapa de práctica administrativa. Por eso <code>aliento_alcoholico</code> no entra a la tipología
La tipología es una discretización, no un descubrimiento	Silueta 0.21 con $k = 4$. Los perfiles son interpretables y estables, pero el espacio es continuo: hay cruces en la frontera entre dos tipos y la asignación de esos es arbitraria
Gi* depende de la banda y del universo	Con <code>banda = 1.5 km</code> y universo «celdas con ≥ 1 accidente». Otra banda da otro número de celdas significativas; el patrón grueso es robusto, el conteo exacto no

La mortalidad no se modela	1,075 fallecimientos en 982 nodos, máximo 4 por nodo. No hay masa para nada por debajo del nivel
----------------------------	--

Límite	Por qué
	municipal

Lo siguiente que desbloquearía análisis, en orden de rendimiento: **marco geoestadístico del INEGI** (contornos, AGEB, población 2020) → tasas y coropletas; **aforos vehiculares** → riesgo real por cruce; **contextily** → mapa base; **libpysal** + **esda** → Moran local y el resto de la caja de herramientas espacial.